

Single-Cell Batch-Correction Evaluation

Contents

Package Setup	2
Configuration	4
Input Seurat Object	5
Normalize And Select Variable Genes	8
Helper Functions	9
Plot Functions	25
Build Representations	29
Technical Batch-Shift Metrics	30
Biological Structure Preservation Metrics	47
Final Method Summary	57
Interpretation Guide	57
Technical Ratio Heatmap	57
Technical Deviation Heatmap	57
Before-vs-after Technical Scatter	57
Cross-vs-within Overlay	58
Alpha-to-beta Relative Distribution	58
Mean Relative Separation Heatmap	58
Biological Preservation Scatter	58
Signed Biological Geometry Difference Heatmap	58
Final Method Summary	58

Package Setup

This chunk is intentionally disabled by `run_package_install <- FALSE`. Only set it to `TRUE` in a fresh R session if packages are missing. If you need to upgrade already-loaded packages such as `dplyr`, restart R first and install before loading `Seurat`, `tidyr`, `plotly`, or `sctransform`.

```
run_package_install <- FALSE

if (run_package_install) {
  install_cran_if_missing <- function(packages) {
    missing_packages <- packages[
      !vapply(packages, requireNamespace, logical(1), quietly = TRUE)
    ]

    if (length(missing_packages) > 0) {
      install.packages(missing_packages)
    }
  }

  install_bioc_if_missing <- function(packages) {
    missing_packages <- packages[
      !vapply(packages, requireNamespace, logical(1), quietly = TRUE)
    ]

    if (length(missing_packages) > 0) {
      BiocManager::install(missing_packages, update = FALSE, ask = FALSE)
    }
  }

  install_cran_if_missing(c(
    "Seurat",
    "dplyr",
    "tidyr",
    "purrr",
    "ggplot2",
    "readr",
    "tibble",
    "future",
    "magrittr",
    "rlang",
    "Matrix",
    "remotes"
  ))

  if (!requireNamespace("BiocManager", quietly = TRUE)) {
    install.packages("BiocManager")
  }

  install_bioc_if_missing(c(
    "SingleCellExperiment",
    "SummarizedExperiment",
    "S4Vectors",
    "batchelor",
    "scran",
  ))
}
```

```

    "scater",
    "BiocSingular",
    "BiocParallel"
  ))

  # Optional, useful if you also want Seurat wrapper methods such as RunFastMNN.
  if (!requireNamespace("SeuratWrappers", quietly = TRUE)) {
    remotes::install_github("satijalab/seurat-wrappers", upgrade = "never")
  }
}

```

```

required_packages <- c(
  "Seurat",
  "SeuratObject",
  "SingleCellExperiment",
  "SummarizedExperiment",
  "S4Vectors",
  "Matrix",
  "batchelor",
  "BiocSingular",
  "BiocParallel",
  "dplyr",
  "magrittr",
  "purrr",
  "ggplot2",
  "readr",
  "tibble",
  "future"
)

missing_packages <- required_packages[
  !vapply(required_packages, requireNamespace, logical(1), quietly = TRUE)
]

if (length(missing_packages) > 0) {
  stop(
    "Missing required packages: ",
    paste(missing_packages, collapse = ", "),
    ". Restart R, install them in a fresh session, then knit again.",
    call. = FALSE
  )
}

# Use namespace-qualified local aliases instead of library(dplyr). This avoids
# dplyr unload/reload conflicts when Seurat dependencies already imported dplyr.
`%>%` <- magrittr::`%>%`
arrange <- dplyr::arrange
bind_rows <- dplyr::bind_rows
filter <- dplyr::filter
full_join <- dplyr::full_join
group_by <- dplyr::group_by
group_split <- dplyr::group_split
inner_join <- dplyr::inner_join

```

```

mutate <- dplyr::mutate
n <- dplyr::n
rename <- dplyr::rename
select <- dplyr::select
summarise <- dplyr::summarise
transmute <- dplyr::transmute

imap <- purrr::imap
map <- purrr::map
map_dfr <- purrr::map_dfr
tibble <- tibble::tibble

aes <- ggplot2::aes
element_blank <- ggplot2::element_blank
element_text <- ggplot2::element_text
facet_grid <- ggplot2::facet_grid
facet_wrap <- ggplot2::facet_wrap
geom_abline <- ggplot2::geom_abline
geom_density <- ggplot2::geom_density
geom_hline <- ggplot2::geom_hline
geom_point <- ggplot2::geom_point
geom_text <- ggplot2::geom_text
geom_tile <- ggplot2::geom_tile
geom_vline <- ggplot2::geom_vline
ggplot <- ggplot2::ggplot
labs <- ggplot2::labs
scale_fill_gradient2 <- ggplot2::scale_fill_gradient2
scale_fill_viridis_c <- ggplot2::scale_fill_viridis_c
theme <- ggplot2::theme
theme_minimal <- ggplot2::theme_minimal
vars <- ggplot2::vars

```

Configuration

Adjust the parameters in the YAML header, or override these values below.

```

batch_col <- params$batch_col
cell_type_col <- params$cell_type_col
assay_name <- params$assay
reference_cell_type <- params$reference_cell_type
beta_cell_type <- params$beta_cell_type
n_dims <- params$n_dims
n_variable_features <- params$n_variable_features
scale_all_genes <- params$scale_all_genes
future_globals_max_size_gb <- params$future_globals_max_size_gb
min_cells_per_group <- params$min_cells_per_group
max_cells_per_group <- params$max_cells_per_group
max_pairs_per_comparison <- params$max_pairs_per_comparison
seed <- params$seed

options(future.globals.maxSize = future_globals_max_size_gb * 1024^3)
future::plan(future::sequential)

```

```

output_dir <- params$output_dir
figures_dir <- file.path(output_dir, "figures")
tables_dir <- file.path(output_dir, "tables")
dir.create(figures_dir, recursive = TRUE, showWarnings = FALSE)
dir.create(tables_dir, recursive = TRUE, showWarnings = FALSE)

```

Input Seurat Object

This report standardizes the input object to `seu`. Use one of these options:

- If your object is already loaded as `seu`, run the report directly.
- If your object has another name, set `seurat_object_name` in the YAML.
- If knitting in a fresh R session, set `seurat_rds` in the YAML to an `.rds` file created with `saveRDS(your_seurat_object, "path/to/file.rds")`.

```

seurat_rds_path <- params$seurat_rds
seurat_object_name <- params$seurat_object_name

# If running chunks interactively, you can temporarily override the YAML here:
# seurat_rds_path <- "/absolute/path/to/your_seurat_object.rds"
# seurat_object_name <- "your_object_name"

find_loaded_seurat_objects <- function() {
  object_names <- ls(envir = .GlobalEnv)
  object_names[
    vapply(
      object_names,
      function(object_name) {
        inherits(get(object_name, envir = .GlobalEnv), "Seurat")
      },
      logical(1)
    )
  ]
}

load_10x_batches_from_root <- function(data_root,
                                       batch_names,
                                       batch_col,
                                       cell_type_col) {
  if (is.null(data_root) || !dir.exists(data_root)) {
    stop(
      "The 10X data root does not exist: ",
      data_root,
      ". Set `tenx_data_root` to the folder containing the batch folders.",
      call. = FALSE
    )
  }

  dataset_dirs <- file.path(data_root, batch_names)
  names(dataset_dirs) <- batch_names

```

```

missing_dirs <- dataset_dirs[!dir.exists(dataset_dirs)]

if (length(missing_dirs) > 0) {
  stop(
    "These 10X batch directories do not exist:\n",
    paste(names(missing_dirs), missing_dirs, sep = ": ", collapse = "\n"),
    call. = FALSE
  )
}

objects <- lapply(names(dataset_dirs), function(batch_name) {
  counts <- Seurat::Read10X(data.dir = dataset_dirs[[batch_name]])

  obj <- Seurat::CreateSeuratObject(
    counts = counts,
    project = batch_name,
    min.cells = 3,
    min.features = 200
  )

  obj[[batch_col]] <- batch_name

  annotation_file <- file.path(dataset_dirs[[batch_name]], "annotations.csv")

  if (file.exists(annotation_file)) {
    annotations <- read.csv(
      annotation_file,
      row.names = "barcode",
      stringsAsFactors = FALSE
    )

    if (!cell_type_col %in% colnames(annotations)) {
      stop(
        "Annotation file for ",
        batch_name,
        " does not contain column `",
        cell_type_col,
        "`.",
        call. = FALSE
      )
    }

    obj[[cell_type_col]] <- annotations[colnames(obj), cell_type_col]
  } else {
    obj[[cell_type_col]] <- NA_character_
    warning("No annotations.csv found for batch: ", batch_name, call. = FALSE)
  }

  obj
})

merged <- if (length(objects) == 1) {
  objects[[1]]
}

```

```

} else {
  merge(
    objects[[1]],
    y = objects[-1],
    add.cell.ids = names(dataset_dirs)
  )
}

if ("JoinLayers" %in% getNamespaceExports("SeuratObject")) {
  merged <- SeuratObject::JoinLayers(merged)
}

merged
}

if (exists("seu", inherits = TRUE) && inherits(seu, "Seurat")) {
  message("Using loaded Seurat object named `seu`.")
} else if (!is.null(seurat_rds_path) && file.exists(seurat_rds_path)) {
  seu <- readRDS(seurat_rds_path)
  message("Loaded Seurat object from: ", seurat_rds_path)
} else if (
  !is.null(seurat_object_name) &&
  exists(seurat_object_name, envir = .GlobalEnv, inherits = FALSE)
) {
  seu <- get(seurat_object_name, envir = .GlobalEnv)
  message("Using loaded Seurat object named `", seurat_object_name, "`.")
} else if (!is.null(params$tenx_data_root) && dir.exists(params$tenx_data_root)) {
  seu <- load_10x_batches_from_root(
    data_root = params$tenx_data_root,
    batch_names = params$tenx_batches,
    batch_col = batch_col,
    cell_type_col = cell_type_col
  )
  message("Loaded 10X batches from: ", params$tenx_data_root)
} else {
  loaded_seurat_objects <- find_loaded_seurat_objects()

  if (length(loaded_seurat_objects) == 1) {
    seu <- get(loaded_seurat_objects[[1]], envir = .GlobalEnv)
    message("Using the only loaded Seurat object: `", loaded_seurat_objects[[1]], "`.")
  } else {
    available_message <- if (length(loaded_seurat_objects) == 0) {
      "No loaded Seurat objects were detected in .GlobalEnv."
    } else {
      paste(
        "Loaded Seurat objects detected:",
        paste(loaded_seurat_objects, collapse = ", ")
      )
    }
  }

  stop(
    "No input Seurat object was found. Load your object as `seu`, ",
    "set `seurat_object_name` to its object name, or set `seurat_rds` ",

```

```

    "to an existing .rds file, or set `tenx_data_root` to the folder ",
    "containing the batch directories. ",
    available_message,
    call. = FALSE
  )
}
}

```

```
## Loaded 10X batches from: /Users/taheralabbar/Desktop/starter code and data/data/panc8
```

```

if (!inherits(seu, "Seurat")) {
  stop("`seu` must be a Seurat object.", call. = FALSE)
}

```

Normalize And Select Variable Genes

This chunk normalizes `seu`, selects the top `n_variable_features` genes by VST for PCA and MNN methods, and scales either all genes or only the selected variable genes depending on `scale_all_genes`.

```

set.seed(seed)
SeuratObject::DefaultAssay(seu) <- assay_name

seu <- Seurat::NormalizeData(seu, verbose = FALSE)
seu <- Seurat::FindVariableFeatures(
  seu,
  selection.method = "vst",
  nfeatures = n_variable_features,
  verbose = FALSE
)

pca_features <- SeuratObject::VariableFeatures(seu)

if (length(pca_features) == 0) {
  stop("No variable features were selected for PCA.", call. = FALSE)
}

features_to_scale <- if (isTRUE(scale_all_genes)) {
  rownames(seu)
} else {
  pca_features
}

seu <- Seurat::ScaleData(
  seu,
  features = features_to_scale,
  verbose = FALSE
)

message(
  "Normalized `seu`; selected ",
  length(pca_features),

```

```

" variable genes for PCA/MNN; scaled ",
length(features_to_scale),
" genes."
)

```

```
## Normalized 'seu'; selected 2000 variable genes for PCA/MNN; scaled 26368 genes.
```

Helper Functions

```

safe_divide <- function(numerator, denominator) {
  out <- numerator / denominator
  bad <- !is.finite(out) | !is.finite(denominator) | denominator <= 0
  out[bad] <- NA_real_
  out
}

safe_abs_log <- function(x) {
  out <- abs(log(x))
  out[!is.finite(out) | x <= 0] <- NA_real_
  out
}

clean_labels <- function(x) {
  x <- as.character(x)
  x[is.na(x) | x == ""] <- NA_character_
  x
}

check_required_metadata <- function(seu, batch_col, cell_type_col, assay_name) {
  metadata <- seu[[ ]]
  missing_cols <- setdiff(c(batch_col, cell_type_col), colnames(metadata))

  if (length(missing_cols) > 0) {
    stop(
      "Missing metadata columns: ",
      paste(missing_cols, collapse = ", "),
      call. = FALSE
    )
  }

  if (!assay_name %in% names(seu@assays)) {
    stop("Assay `", assay_name, "` was not found in `seu`.", call. = FALSE)
  }

  metadata[[batch_col]] <- clean_labels(metadata[[batch_col]])
  metadata[[cell_type_col]] <- clean_labels(metadata[[cell_type_col]])

  if (sum(!is.na(metadata[[batch_col]])) < 2) {
    stop("The batch column has fewer than two non-missing cells.", call. = FALSE)
  }
}

```

```

if (sum(!is.na(metadata[[cell_type_col]])) < 2) {
  stop("The cell-type column has fewer than two non-missing cells.", call. = FALSE)
}

metadata
}

sample_vector <- function(x, max_n) {
  x <- as.character(x)

  if (is.null(max_n) || is.na(max_n) || length(x) <= max_n) {
    return(x)
  }

  sample(x, max_n)
}

effective_rank <- function(requested_rank, n_cells, n_features) {
  max(1L, min(as.integer(requested_rank), as.integer(n_cells) - 1L, as.integer(n_features)))
}

get_seurat_layer <- function(seu, assay_name, layer_name) {
  tryCatch(
    SeuratObject::LayerData(seu, assay = assay_name, layer = layer_name),
    error = function(e) {
      SeuratObject::GetAssayData(seu, assay = assay_name, slot = layer_name)
    }
  )
}

make_sce_from_seurat <- function(seu, assay_name) {
  logcounts_mat <- get_seurat_layer(seu, assay_name, "data")

  if (nrow(logcounts_mat) == 0 || ncol(logcounts_mat) == 0) {
    stop(
      "No normalized data layer was found. Run NormalizeData() before MNN methods.",
      call. = FALSE
    )
  }

  assays <- list(logcounts = logcounts_mat)

  counts_mat <- tryCatch(
    get_seurat_layer(seu, assay_name, "counts"),
    error = function(e) NULL
  )

  if (!is.null(counts_mat) && identical(dim(counts_mat), dim(logcounts_mat))) {
    assays$counts <- counts_mat[rownames(logcounts_mat), colnames(logcounts_mat), drop = FALSE]
  }

  sce <- SingleCellExperiment::SingleCellExperiment(assays = assays)
  SummarizedExperiment::colData(sce) <- S4Vectors::DataFrame(

```

```

    seu[[ ]][colnames(sce), , drop = FALSE]
  )

  sce
}

align_embedding <- function(embedding, metadata, method_name) {
  embedding <- as.matrix(embedding)

  if (is.null(rownames(embedding))) {
    stop("Embedding for ", method_name, " has no cell rownames.", call. = FALSE)
  }

  if (anyDuplicated(rownames(embedding)) > 0) {
    stop("Embedding for ", method_name, " has duplicated cell rownames.", call. = FALSE)
  }

  missing_cells <- setdiff(rownames(metadata), rownames(embedding))

  if (length(missing_cells) > 0) {
    stop(
      "Embedding for ",
      method_name,
      " is missing ",
      length(missing_cells),
      " metadata cells. First missing cell: ",
      missing_cells[[1]],
      call. = FALSE
    )
  }

  embedding[rownames(metadata), , drop = FALSE]
}

run_uncorrected_pca_embedding <- function(seu,
                                           assay_name,
                                           n_dims,
                                           n_variable_features,
                                           seed) {
  set.seed(seed)
  SeuratObject::DefaultAssay(seu) <- assay_name

  features <- SeuratObject::VariableFeatures(seu)

  if (length(features) == 0) {
    seu <- Seurat::NormalizeData(seu, verbose = FALSE)
    seu <- Seurat::FindVariableFeatures(
      seu,
      selection.method = "vst",
      nfeatures = n_variable_features,
      verbose = FALSE
    )
    features <- SeuratObject::VariableFeatures(seu)
  }

```

```

}

features <- head(features, n_variable_features)
rank <- effective_rank(n_dims, ncol(seu), length(features))

scale_data <- tryCatch(
  get_seurat_layer(seu, assay_name, "scale.data"),
  error = function(e) NULL
)

missing_scaled_features <- if (is.null(scale_data) || nrow(scale_data) == 0) {
  features
} else {
  setdiff(features, rownames(scale_data))
}

if (length(missing_scaled_features) > 0) {
  seu <- Seurat::ScaleData(seu, features = features, verbose = FALSE)
}

seu <- Seurat::RunPCA(
  seu,
  features = features,
  npcs = rank,
  reduction.name = "pca_uncorrected",
  seed.use = seed,
  verbose = FALSE
)

embedding <- SeuratObject::Embeddings(seu, reduction = "pca_uncorrected")
embedding <- embedding[, seq_len(rank), drop = FALSE]

list(
  object = seu,
  embedding = embedding,
  features = features
)
}

run_fastmnn_embedding <- function(seu,
                                  metadata,
                                  batch_col,
                                  assay_name,
                                  features,
                                  n_dims,
                                  seed,
                                  k = 20) {
  set.seed(seed)
  sce <- make_sce_from_seurat(seu, assay_name)
  batch <- factor(metadata[colnames(sce), batch_col])
  features <- intersect(features, rownames(sce))
  rank <- effective_rank(n_dims, ncol(sce), length(features))

```

```

fastmnn_out <- batchelor::fastMNN(
  sce,
  batch = batch,
  subset.row = features,
  d = rank,
  k = k,
  assay.type = "logcounts",
  BPPARAM = BiocParallel::SerialParam()
)

embedding <- SingleCellExperiment::reducedDim(fastmnn_out, "corrected")
rownames(embedding) <- colnames(fastmnn_out)
colnames(embedding) <- paste0("FastMNN_", seq_len(ncol(embedding)))
embedding
}

run_mnn_embedding <- function(seu,
                             metadata,
                             batch_col,
                             assay_name,
                             features,
                             n_dims,
                             seed,
                             k = 20,
                             sigma = 0.1) {
  set.seed(seed)
  sce <- make_sce_from_seurat(seu, assay_name)
  batch <- factor(metadata[colnames(sce), batch_col])
  features <- intersect(features, rownames(sce))

  mnn_out <- batchelor::mnnCorrect(
    sce,
    batch = batch,
    subset.row = features,
    k = k,
    sigma = sigma,
    assay.type = "logcounts",
    correct.all = FALSE,
    BPPARAM = BiocParallel::SerialParam()
  )

  corrected <- SummarizedExperiment::assay(mnn_out, "corrected")
  rank <- effective_rank(n_dims, ncol(corrected), nrow(corrected))

  pca <- BiocSingular::runPCA(
    t(corrected),
    rank = rank,
    center = TRUE,
    scale = FALSE,
    BSPARAM = BiocSingular::IrlbaParam(),
    BPPARAM = BiocParallel::SerialParam()
  )
}

```

```

embedding <- pca$x
rownames(embedding) <- colnames(corrected)
colnames(embedding) <- paste0("MNN_PC_", seq_len(ncol(embedding)))
embedding
}

run_seurat_cca_embedding <- function(seu,
                                     batch_col,
                                     assay_name,
                                     n_dims,
                                     n_variable_features,
                                     seed) {
  set.seed(seed)
  future::plan(future::sequential)
  SeuratObject::DefaultAssay(seu) <- assay_name

  object_list <- Seurat::SplitObject(seu, split.by = batch_col)
  object_list <- object_list[vapply(object_list, ncol, numeric(1)) > 1]

  if (length(object_list) < 2) {
    stop("Seurat CCA requires at least two batches with more than one cell.", call. = FALSE)
  }

  object_list <- lapply(object_list, function(obj) {
    SeuratObject::DefaultAssay(obj) <- assay_name
    obj <- Seurat::NormalizeData(obj, verbose = FALSE)
    Seurat::FindVariableFeatures(
      obj,
      selection.method = "vst",
      nfeatures = n_variable_features,
      verbose = FALSE
    )
  })

  features <- Seurat::SelectIntegrationFeatures(
    object_list = object_list,
    nfeatures = n_variable_features
  )

  min_batch_cells <- min(vapply(object_list, ncol, numeric(1)))
  rank <- effective_rank(n_dims, min_batch_cells, length(features))
  dims <- seq_len(rank)

  object_list <- lapply(object_list, function(obj) {
    Seurat::ScaleData(obj, features = features, verbose = FALSE)
  })

  k_anchor <- min(5L, min_batch_cells - 1L)
  k_filter <- min(200L, min_batch_cells - 1L)
  k_score <- min(30L, min_batch_cells - 1L)
  k_weight <- min(100L, min_batch_cells - 1L)

  anchors <- Seurat::FindIntegrationAnchors(

```

```

    object.list = object_list,
    anchor.features = features,
    reduction = "cca",
    dims = dims,
    k.anchor = k_anchor,
    k.filter = k_filter,
    k.score = k_score,
    verbose = FALSE
  )

  integrated <- Seurat::IntegrateData(
    anchorset = anchors,
    new.assay.name = "integrated",
    dims = dims,
    k.weight = k_weight,
    verbose = FALSE
  )

  SeuratObject::DefaultAssay(integrated) <- "integrated"
  integrated <- Seurat::ScaleData(integrated, verbose = FALSE)
  integrated <- Seurat::RunPCA(
    integrated,
    npcs = rank,
    reduction.name = "integrated_pca",
    seed.use = seed,
    verbose = FALSE
  )

  embedding <- SeuratObject::Embeddings(integrated, reduction = "integrated_pca")
  embedding <- embedding[, seq_len(rank), drop = FALSE]
  colnames(embedding) <- paste0("SeuratCCA_PC_", seq_len(ncol(embedding)))

  list(
    object = integrated,
    embedding = embedding,
    features = features
  )
}

make_between_pairs <- function(cells_x, cells_y, max_pairs) {
  n_pairs_total <- length(cells_x) * length(cells_y)
  n_to_sample <- min(max_pairs, n_pairs_total)

  if (n_pairs_total <= max_pairs) {
    grid <- expand.grid(
      cell_x = cells_x,
      cell_y = cells_y,
      KEEP.OUT.ATTRS = FALSE,
      stringsAsFactors = FALSE
    )
    return(grid)
  }
}

```

```

tibble(
  cell_x = sample(cells_x, n_to_sample, replace = TRUE),
  cell_y = sample(cells_y, n_to_sample, replace = TRUE)
)
}

make_within_pairs <- function(cells, max_pairs) {
  if (length(cells) < 2) {
    return(tibble(cell_x = character(), cell_y = character()))
  }

  n_pairs_total <- length(cells) * (length(cells) - 1) / 2
  n_to_sample <- min(max_pairs, n_pairs_total)

  if (n_pairs_total <= max_pairs) {
    pair_matrix <- utils::combn(cells, 2)
    return(tibble(cell_x = pair_matrix[1, ], cell_y = pair_matrix[2, ]))
  }

  cell_x <- character()
  cell_y <- character()

  while (length(cell_x) < n_to_sample) {
    remaining <- n_to_sample - length(cell_x)
    candidate_x <- sample(cells, remaining * 2, replace = TRUE)
    candidate_y <- sample(cells, remaining * 2, replace = TRUE)
    keep <- candidate_x != candidate_y

    cell_x <- c(cell_x, candidate_x[keep])
    cell_y <- c(cell_y, candidate_y[keep])
  }

  tibble(
    cell_x = cell_x[seq_len(n_to_sample)],
    cell_y = cell_y[seq_len(n_to_sample)]
  )
}

paired_euclidean <- function(embedding, cell_x, cell_y) {
  x <- embedding[cell_x, , drop = FALSE]
  y <- embedding[cell_y, , drop = FALSE]
  sqrt(rowSums((x - y)^2))
}

sample_between_distances <- function(embedding, cells_x, cells_y, max_pairs) {
  pairs <- make_between_pairs(cells_x, cells_y, max_pairs)
  pairs$distance <- paired_euclidean(embedding, pairs$cell_x, pairs$cell_y)
  pairs
}

sample_within_distances <- function(embedding, cells, max_pairs) {
  pairs <- make_within_pairs(cells, max_pairs)

```

```

if (nrow(pairs) == 0) {
  pairs$distance <- numeric()
  return(pairs)
}

pairs$distance <- paired_euclidean(embedding, pairs$cell_x, pairs$cell_y)
pairs
}

cells_for_group <- function(metadata,
                             batch_col,
                             cell_type_col,
                             batch,
                             cell_type,
                             max_cells_per_group) {
  cells <- rownames(metadata)[
    metadata[[batch_col]] == batch &
    metadata[[cell_type_col]] == cell_type &
    !is.na(metadata[[batch_col]]) &
    !is.na(metadata[[cell_type_col]])
  ]

  sample_vector(cells, max_cells_per_group)
}

compute_technical_metrics_for_embedding <- function(embedding,
                                                    metadata,
                                                    batch_col,
                                                    cell_type_col,
                                                    method,
                                                    min_cells_per_group,
                                                    max_cells_per_group,
                                                    max_pairs_per_comparison,
                                                    seed) {
  set.seed(seed)
  metadata <- metadata[rownames(embedding), , drop = FALSE]
  metadata[[batch_col]] <- clean_labels(metadata[[batch_col]])
  metadata[[cell_type_col]] <- clean_labels(metadata[[cell_type_col]])

  batches <- sort(unique(stats::na.omit(metadata[[batch_col]])))
  cell_types <- sort(unique(stats::na.omit(metadata[[cell_type_col]])))
  batch_pairs <- utils::combn(batches, 2, simplify = FALSE)

  summary_rows <- list()
  distance_rows <- list()
  row_index <- 1L
  distance_index <- 1L

  for (batch_pair in batch_pairs) {
    batch_1 <- batch_pair[[1]]
    batch_2 <- batch_pair[[2]]
    batch_pair_label <- paste(batch_1, batch_2, sep = " vs ")

```

```

for (cell_type in cell_types) {
  cells_1 <- cells_for_group(
    metadata,
    batch_col,
    cell_type_col,
    batch_1,
    cell_type,
    max_cells_per_group
  )
  cells_2 <- cells_for_group(
    metadata,
    batch_col,
    cell_type_col,
    batch_2,
    cell_type,
    max_cells_per_group
  )

  if (length(cells_1) < min_cells_per_group || length(cells_2) < min_cells_per_group) {
    message(
      "Skipping technical metric for ",
      method,
      ", ",
      cell_type,
      ", ",
      batch_pair_label,
      ": too few cells."
    )
    next
  }

  cross_distances <- sample_between_distances(
    embedding,
    cells_1,
    cells_2,
    max_pairs_per_comparison
  )
  within_1 <- sample_within_distances(embedding, cells_1, max_pairs_per_comparison)
  within_2 <- sample_within_distances(embedding, cells_2, max_pairs_per_comparison)

  if (nrow(within_1) == 0 || nrow(within_2) == 0) {
    message(
      "Skipping technical metric for ",
      method,
      ", ",
      cell_type,
      ", ",
      batch_pair_label,
      ": no within-batch distances."
    )
    next
  }
}

```

```

within_baseline <- median(c(within_1$distance, within_2$distance), na.rm = TRUE)
median_cross <- median(cross_distances$distance, na.rm = TRUE)
r_tech <- safe_divide(median_cross, within_baseline)

summary_rows[[row_index]] <- tibble(
  method = method,
  batch_1 = batch_1,
  batch_2 = batch_2,
  batch_pair = batch_pair_label,
  cell_type = cell_type,
  n_cells_batch_1 = length(cells_1),
  n_cells_batch_2 = length(cells_2),
  n_cross_distances = nrow(cross_distances),
  n_within_distances_batch_1 = nrow(within_1),
  n_within_distances_batch_2 = nrow(within_2),
  median_cross_batch_distance = median_cross,
  within_baseline = within_baseline,
  R_tech = r_tech,
  tech_deviation = safe_abs_log(r_tech)
)
row_index <- row_index + 1L

distance_rows[[distance_index]] <- bind_rows(
  cross_distances %>%
    transmute(distance = distance) %>%
    mutate(distance_type = "Cross batch", source_batch = NA_character_),
  within_1 %>%
    transmute(distance = distance) %>%
    mutate(distance_type = "Within batch", source_batch = batch_1),
  within_2 %>%
    transmute(distance = distance) %>%
    mutate(distance_type = "Within batch", source_batch = batch_2)
) %>%
  mutate(
    method = method,
    batch_1 = batch_1,
    batch_2 = batch_2,
    batch_pair = batch_pair_label,
    cell_type = cell_type,
    within_baseline = within_baseline,
    normalized_distance = safe_divide(distance, within_baseline),
    .before = 1
  )
distance_index <- distance_index + 1L
}
}

list(
  summary = bind_rows(summary_rows),
  distances = bind_rows(distance_rows)
)
}

```

```

compute_biological_metrics_for_embedding <- function(embedding,
                                                    metadata,
                                                    batch_col,
                                                    cell_type_col,
                                                    method,
                                                    reference_cell_type,
                                                    min_cells_per_group,
                                                    max_cells_per_group,
                                                    max_pairs_per_comparison,
                                                    seed) {

  set.seed(seed)
  metadata <- metadata[rownames(embedding), , drop = FALSE]
  metadata[[batch_col]] <- clean_labels(metadata[[batch_col]])
  metadata[[cell_type_col]] <- clean_labels(metadata[[cell_type_col]])

  batches <- sort(unique(stats::na.omit(metadata[[batch_col]])))

  summary_rows <- list()
  distance_rows <- list()
  row_index <- 1L
  distance_index <- 1L

  for (batch in batches) {
    batch_metadata <- metadata[
      metadata[[batch_col]] == batch &
        !is.na(metadata[[cell_type_col]]),
      ,
      drop = FALSE
    ]

    alpha_cells <- rownames(batch_metadata)[
      batch_metadata[[cell_type_col]] == reference_cell_type
    ]
    alpha_cells <- sample_vector(alpha_cells, max_cells_per_group)

    if (length(alpha_cells) < max(2L, min_cells_per_group)) {
      message(
        "Skipping biological metric for ",
        method,
        ", ",
        batch,
        ": fewer than ",
        max(2L, min_cells_per_group),
        " ",
        reference_cell_type,
        " cells."
      )
      next
    }

    alpha_within <- sample_within_distances(
      embedding,
      alpha_cells,

```

```

    max_pairs_per_comparison
  )
  alpha_baseline <- median(alpha_within$distance, na.rm = TRUE)

  if (!is.finite(alpha_baseline) || alpha_baseline <= 0) {
    message(
      "Skipping biological metric for ",
      method,
      ", ",
      batch,
      ": invalid alpha baseline."
    )
    next
  }

  target_cell_types <- sort(unique(stats::na.omit(batch_metadata[[cell_type_col]])))

  for (target_cell_type in target_cell_types) {
    target_cells <- rownames(batch_metadata)[
      batch_metadata[[cell_type_col]] == target_cell_type
    ]

    if (target_cell_type == reference_cell_type) {
      distances <- alpha_within
      n_target_cells <- length(alpha_cells)
    } else {
      target_cells <- sample_vector(target_cells, max_cells_per_group)

      if (length(target_cells) < min_cells_per_group) {
        message(
          "Skipping biological metric for ",
          method,
          ", ",
          batch,
          ", ",
          reference_cell_type,
          " vs ",
          target_cell_type,
          ": too few target cells."
        )
        next
      }

      distances <- sample_between_distances(
        embedding,
        alpha_cells,
        target_cells,
        max_pairs_per_comparison
      )
      n_target_cells <- length(target_cells)
    }

    distances <- distances %>%

```

```

      mutate(relative_distance = safe_divide(distance, alpha_baseline))

summary_rows[[row_index]] <- tibble(
  method = method,
  batch = batch,
  reference_cell_type = reference_cell_type,
  target_cell_type = target_cell_type,
  n_reference_cells = length(alpha_cells),
  n_target_cells = n_target_cells,
  n_pairwise_distances = nrow(distances),
  alpha_baseline = alpha_baseline,
  mean_relative_separation = mean(distances$relative_distance, na.rm = TRUE),
  median_relative_separation = median(distances$relative_distance, na.rm = TRUE)
)
row_index <- row_index + 1L

distance_rows[[distance_index]] <- distances %>%
  transmute(
    method = method,
    batch = batch,
    reference_cell_type = reference_cell_type,
    target_cell_type = target_cell_type,
    alpha_baseline = alpha_baseline,
    distance = distance,
    relative_distance = relative_distance
  )
distance_index <- distance_index + 1L
}
}

list(
  summary = bind_rows(summary_rows),
  distances = bind_rows(distance_rows)
)
}

compute_biological_geometry_delta <- function(biological_summary) {
  split_groups <- biological_summary %>%
    group_by(method, target_cell_type) %>%
    group_split()

  map_dfr(split_groups, function(group_df) {
    batches <- sort(unique(group_df$batch))

    if (length(batches) < 2) {
      return(tibble())
    }

    map_dfr(utils::combn(batches, 2, simplify = FALSE), function(batch_pair) {
      batch_1 <- batch_pair[[1]]
      batch_2 <- batch_pair[[2]]

      value_1 <- group_df$mean_relative_separation[group_df$batch == batch_1][[1]]

```

```

value_2 <- group_df$mean_relative_separation[group_df$batch == batch_2][[1]]

tibble(
  method = group_df$method[[1]],
  target_cell_type = group_df$target_cell_type[[1]],
  batch_1 = batch_1,
  batch_2 = batch_2,
  batch_pair = paste(batch_1, batch_2, sep = " vs "),
  mean_relative_batch_1 = value_1,
  mean_relative_batch_2 = value_2,
  signed_delta = value_1 - value_2
)
})
})
}

make_technical_scatter_data <- function(technical_summary,
                                       baseline_method = "Uncorrected_PCA") {
  baseline <- technical_summary %>%
    filter(method == baseline_method) %>%
    select(batch_pair, cell_type, R_tech_before = R_tech)

  technical_summary %>%
    filter(method != baseline_method) %>%
    inner_join(baseline, by = c("batch_pair", "cell_type")) %>%
    rename(R_tech_after = R_tech)
}

make_biological_scatter_data <- function(biological_summary,
                                       baseline_method = "Uncorrected_PCA") {
  baseline <- biological_summary %>%
    filter(method == baseline_method) %>%
    select(
      batch,
      target_cell_type,
      mean_relative_before = mean_relative_separation
    )

  biological_summary %>%
    filter(method != baseline_method) %>%
    inner_join(baseline, by = c("batch", "target_cell_type")) %>%
    rename(mean_relative_after = mean_relative_separation)
}

make_final_method_summary <- function(technical_summary,
                                       biological_summary,
                                       biological_delta,
                                       beta_cell_type,
                                       baseline_method = "Uncorrected_PCA") {
  technical_scores <- technical_summary %>%
    group_by(method) %>%
    summarise(
      technical_score = median(tech_deviation, na.rm = TRUE),

```

```

    median_R_tech = median(R_tech, na.rm = TRUE),
    n_technical_comparisons = n(),
    .groups = "drop"
)

baseline_bio <- biological_summary %>%
  filter(method == baseline_method) %>%
  select(
    batch,
    target_cell_type,
    mean_relative_before = mean_relative_separation
  )

biological_preservation_scores <- biological_summary %>%
  inner_join(baseline_bio, by = c("batch", "target_cell_type")) %>%
  mutate(
    relative_to_uncorrected = safe_divide(
      mean_relative_separation,
      mean_relative_before
    ),
    biological_preservation_error = safe_abs_log(relative_to_uncorrected)
  ) %>%
  group_by(method) %>%
  summarise(
    biological_preservation_score = median(
      biological_preservation_error,
      na.rm = TRUE
    ),
    n_biological_comparisons = n(),
    .groups = "drop"
  )

batch_agreement_scores <- biological_delta %>%
  group_by(method) %>%
  summarise(
    biological_batch_agreement_score = median(abs(signed_delta), na.rm = TRUE),
    n_biological_batch_pairs = n(),
    .groups = "drop"
  )

alpha_beta_scores <- biological_summary %>%
  filter(target_cell_type == beta_cell_type) %>%
  group_by(method) %>%
  summarise(
    alpha_beta_collapse_risk = median(mean_relative_separation, na.rm = TRUE),
    n_beta_batches = n(),
    .groups = "drop"
  )

technical_scores %>%
  full_join(biological_preservation_scores, by = "method") %>%
  full_join(batch_agreement_scores, by = "method") %>%
  full_join(alpha_beta_scores, by = "method") %>%

```

```

    arrange(technical_score)
  }

filter_overlay_distances <- function(distance_table,
                                     selected_batch_pairs = NULL,
                                     selected_cell_types = NULL) {

  out <- distance_table

  if (!is.null(selected_batch_pairs)) {
    out <- out %>% filter(batch_pair %in% selected_batch_pairs)
  }

  if (!is.null(selected_cell_types)) {
    out <- out %>% filter(cell_type %in% selected_cell_types)
  }

  out
}

save_plot <- function(plot, filename, width = 10, height = 7, dpi = 300) {
  ggplot2::ggsave(
    filename = file.path(figures_dir, filename),
    plot = plot,
    width = width,
    height = height,
    dpi = dpi
  )
}

```

Plot Functions

```

plot_technical_ratio_heatmap <- function(technical_summary) {
  ggplot(
    technical_summary,
    aes(x = batch_pair, y = cell_type, fill = R_tech)
  ) +
  geom_tile(color = "white", linewidth = 0.35) +
  geom_text(aes(label = round(R_tech, 2)), size = 2.8) +
  facet_wrap(vars(method)) +
  scale_fill_gradient2(
    low = "#2B6CB0",
    mid = "white",
    high = "#C2410C",
    midpoint = 1,
    na.value = "grey90"
  ) +
  labs(
    title = "Technical ratio by cell type and batch pair",
    subtitle = "R_tech = median cross-batch same-cell-type distance / within-batch baseline",
    x = "Batch pair",
    y = "Cell type",
  )
}

```

```

    fill = "R_tech"
  ) +
  theme_minimal(base_size = 11) +
  theme(
    panel.grid = element_blank(),
    axis.text.x = element_text(angle = 45, hjust = 1)
  )
}

plot_technical_deviation_heatmap <- function(technical_summary) {
  ggplot(
    technical_summary,
    aes(x = batch_pair, y = cell_type, fill = tech_deviation)
  ) +
  geom_tile(color = "white", linewidth = 0.35) +
  geom_text(aes(label = round(tech_deviation, 2)), size = 2.8) +
  facet_wrap(vars(method)) +
  scale_fill_viridis_c(option = "C", direction = -1, na.value = "grey90") +
  labs(
    title = "Technical deviation from ideal mixing",
    subtitle = "Lower abs(log(R_tech)) is better",
    x = "Batch pair",
    y = "Cell type",
    fill = "abs(log(R_tech))"
  ) +
  theme_minimal(base_size = 11) +
  theme(
    panel.grid = element_blank(),
    axis.text.x = element_text(angle = 45, hjust = 1)
  )
}

plot_technical_scatter <- function(technical_scatter) {
  ggplot(
    technical_scatter,
    aes(x = R_tech_before, y = R_tech_after, color = cell_type)
  ) +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed", color = "grey45") +
  geom_hline(yintercept = 1, linetype = "dotted", color = "grey25") +
  geom_point(size = 2.2, alpha = 0.85) +
  facet_wrap(vars(method)) +
  labs(
    title = "Before-vs-after technical ratio",
    subtitle = "Each point is one cell type and batch pair",
    x = "Uncorrected PCA R_tech",
    y = "Corrected R_tech",
    color = "Cell type"
  ) +
  theme_minimal(base_size = 11)
}

plot_cross_vs_within_overlay <- function(distance_table) {
  ggplot(

```

```

distance_table,
aes(x = normalized_distance, fill = distance_type, color = distance_type)
) +
geom_density(alpha = 0.25, linewidth = 0.5, na.rm = TRUE) +
geom_vline(xintercept = 1, linetype = "dashed", color = "grey35") +
facet_grid(cell_type ~ batch_pair ~ method, scales = "free_y") +
labs(
  title = "Cross-batch and within-batch normalized distance distributions",
  subtitle = "Distances are divided by the within-batch same-cell-type baseline inside each representation",
  x = "Normalized distance",
  y = "Density",
  fill = "Distance type",
  color = "Distance type"
) +
theme_minimal(base_size = 10)
}

plot_alpha_beta_distribution <- function(biological_distances, beta_cell_type) {
  biological_distances %>%
    filter(target_cell_type == beta_cell_type) %>%
    ggplot(aes(x = relative_distance, color = method, fill = method)) +
    geom_density(alpha = 0.18, linewidth = 0.55, na.rm = TRUE) +
    geom_vline(xintercept = 1, linetype = "dashed", color = "grey35") +
    facet_wrap(vars(batch), scales = "free_y") +
    labs(
      title = paste("Relative alpha-to-", beta_cell_type, " distance distributions", sep = ""),
      subtitle = "Values mostly above 1 indicate separation from the alpha baseline",
      x = "Pairwise alpha-to-target distance / alpha-within-batch baseline",
      y = "Density",
      color = "Method",
      fill = "Method"
    ) +
    theme_minimal(base_size = 11)
}

plot_mean_relative_heatmap <- function(biological_summary) {
  ggplot(
    biological_summary,
    aes(x = batch, y = target_cell_type, fill = mean_relative_separation)
  ) +
  geom_tile(color = "white", linewidth = 0.35) +
  geom_text(aes(label = round(mean_relative_separation, 2)), size = 2.8) +
  facet_wrap(vars(method)) +
  scale_fill_viridis_c(option = "D", na.value = "grey90") +
  labs(
    title = "Mean alpha-centered relative separation",
    subtitle = "Mean of pairwise relative distances, not a ratio of medians",
    x = "Batch",
    y = "Target cell type",
    fill = "Mean relative\nseparation"
  ) +
  theme_minimal(base_size = 11) +
  theme(panel.grid = element_blank())
}

```

```

}

plot_biological_scatter <- function(biological_scatter) {
  ggplot(
    biological_scatter,
    aes(
      x = mean_relative_before,
      y = mean_relative_after,
      color = target_cell_type
    )
  ) +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed", color = "grey45") +
  geom_hline(yintercept = 1, linetype = "dotted", color = "grey25") +
  geom_point(size = 2.2, alpha = 0.85) +
  facet_wrap(vars(method)) +
  labs(
    title = "Biological preservation relative to uncorrected PCA",
    subtitle = "Movement toward 1 can indicate biological collapse",
    x = "Uncorrected PCA mean relative separation",
    y = "Corrected mean relative separation",
    color = "Target cell type"
  ) +
  theme_minimal(base_size = 11)
}

plot_signed_delta_heatmap <- function(biological_delta) {
  ggplot(
    biological_delta,
    aes(x = batch_pair, y = target_cell_type, fill = signed_delta)
  ) +
  geom_tile(color = "white", linewidth = 0.35) +
  geom_text(aes(label = round(signed_delta, 2)), size = 2.8) +
  facet_wrap(vars(method)) +
  scale_fill_gradient2(
    low = "#2B6CB0",
    mid = "white",
    high = "#C2410C",
    midpoint = 0,
    na.value = "grey90"
  ) +
  labs(
    title = "Signed biological geometry difference across batches",
    subtitle = "Delta = mean relative separation in batch 1 minus batch 2",
    x = "Batch pair",
    y = "Target cell type",
    fill = "Signed delta"
  ) +
  theme_minimal(base_size = 11) +
  theme(
    panel.grid = element_blank(),
    axis.text.x = element_text(angle = 45, hjust = 1)
  )
}

```

Build Representations

All distance comparisons below are made inside a representation and then normalized inside that same representation. The code does not compare raw original-space distances directly against corrected embedding distances.

```
metadata <- check_required_metadata(seu, batch_col, cell_type_col, assay_name)

pca_result <- run_uncorrected_pca_embedding(
  seu = seu,
  assay_name = assay_name,
  n_dims = n_dims,
  n_variable_features = n_variable_features,
  seed = seed
)

metadata <- check_required_metadata(pca_result$object, batch_col, cell_type_col, assay_name)
metadata <- metadata[colnames(pca_result$object), , drop = FALSE]

fastmnn_embedding <- run_fastmnn_embedding(
  seu = pca_result$object,
  metadata = metadata,
  batch_col = batch_col,
  assay_name = assay_name,
  features = pca_result$features,
  n_dims = n_dims,
  seed = seed + 1
)

mnn_embedding <- run_mnn_embedding(
  seu = pca_result$object,
  metadata = metadata,
  batch_col = batch_col,
  assay_name = assay_name,
  features = pca_result$features,
  n_dims = n_dims,
  seed = seed + 2
)

cca_result <- run_seurat_cca_embedding(
  seu = seu,
  batch_col = batch_col,
  assay_name = assay_name,
  n_dims = n_dims,
  n_variable_features = n_variable_features,
  seed = seed + 3
)

## Warning: Different features in new layer data than already exists for
## scale.data
## Warning: Different features in new layer data than already exists for
## scale.data
## Warning: Different features in new layer data than already exists for
```

```
## scale.data
## Warning: Different features in new layer data than already exists for
## scale.data
## Warning: Different features in new layer data than already exists for
## scale.data

## Warning: Layer counts isn't present in the assay object; returning NULL
## Warning: Layer counts isn't present in the assay object; returning NULL
## Warning: Layer counts isn't present in the assay object; returning NULL

## Warning: Different cells in new layer data than already exists for scale.data

## Warning: Layer counts isn't present in the assay object; returning NULL
```

```
embeddings <- list(
  Uncorrected_PCA = pca_result$embedding,
  FastMNN = fastmnn_embedding,
  MNN = mnn_embedding,
  Seurat_CCA = cca_result$embedding
)

embeddings <- imap(embeddings, ~ align_embedding(.x, metadata, .y))

embedding_dimensions <- tibble(
  method = names(embeddings),
  n_cells = vapply(embeddings, nrow, numeric(1)),
  n_dimensions = vapply(embeddings, ncol, numeric(1))
)

embedding_dimensions
```

```
## # A tibble: 4 x 3
##   method      n_cells n_dimensions
##   <chr>      <dbl>      <dbl>
## 1 Uncorrected_PCA  14890         30
## 2 FastMNN        14890         30
## 3 MNN            14890         30
## 4 Seurat_CCA     14890         30
```

Technical Batch-Shift Metrics

For each method, cell type, and batch pair:

```
R_tech = median cross-batch same-cell-type distance /
          median within-batch same-cell-type baseline
```

The target is `R_tech` near 1. The score `abs(log(R_tech))` is lower when the ratio is closer to 1.

```

technical_results <- imap(embeddings, function(embedding, method_name) {
  compute_technical_metrics_for_embedding(
    embedding = embedding,
    metadata = metadata,
    batch_col = batch_col,
    cell_type_col = cell_type_col,
    method = method_name,
    min_cells_per_group = min_cells_per_group,
    max_cells_per_group = max_cells_per_group,
    max_pairs_per_comparison = max_pairs_per_comparison,
    seed = seed
  )
})

```

```

## Skipping technical metric for Uncorrected_PCA, endothelial, celseq vs celseq2: too few cells.

## Skipping technical metric for Uncorrected_PCA, epsilon, celseq vs celseq2: too few cells.

## Skipping technical metric for Uncorrected_PCA, macrophage, celseq vs celseq2: too few cells.

## Skipping technical metric for Uncorrected_PCA, mast, celseq vs celseq2: too few cells.

## Skipping technical metric for Uncorrected_PCA, quiescent_stellate, celseq vs celseq2: too few cells.

## Skipping technical metric for Uncorrected_PCA, schwann, celseq vs celseq2: too few cells.

## Skipping technical metric for Uncorrected_PCA, endothelial, celseq vs fluidigm1: too few cells.

## Skipping technical metric for Uncorrected_PCA, epsilon, celseq vs fluidigm1: too few cells.

## Skipping technical metric for Uncorrected_PCA, macrophage, celseq vs fluidigm1: too few cells.

## Skipping technical metric for Uncorrected_PCA, mast, celseq vs fluidigm1: too few cells.

## Skipping technical metric for Uncorrected_PCA, quiescent_stellate, celseq vs fluidigm1: too few cells.

## Skipping technical metric for Uncorrected_PCA, schwann, celseq vs fluidigm1: too few cells.

## Skipping technical metric for Uncorrected_PCA, endothelial, celseq vs indrop: too few cells.

## Skipping technical metric for Uncorrected_PCA, epsilon, celseq vs indrop: too few cells.

## Skipping technical metric for Uncorrected_PCA, macrophage, celseq vs indrop: too few cells.

## Skipping technical metric for Uncorrected_PCA, mast, celseq vs indrop: too few cells.

## Skipping technical metric for Uncorrected_PCA, quiescent_stellate, celseq vs indrop: too few cells.

## Skipping technical metric for Uncorrected_PCA, schwann, celseq vs indrop: too few cells.

```

Skipping technical metric for Uncorrected_PCA, endothelial, celseq vs smartseq2: too few cells.

Skipping technical metric for Uncorrected_PCA, epsilon, celseq vs smartseq2: too few cells.

Skipping technical metric for Uncorrected_PCA, macrophage, celseq vs smartseq2: too few cells.

Skipping technical metric for Uncorrected_PCA, mast, celseq vs smartseq2: too few cells.

Skipping technical metric for Uncorrected_PCA, quiescent_stellate, celseq vs smartseq2: too few cells.

Skipping technical metric for Uncorrected_PCA, schwann, celseq vs smartseq2: too few cells.

Skipping technical metric for Uncorrected_PCA, epsilon, celseq2 vs fluidigm1: too few cells.

Skipping technical metric for Uncorrected_PCA, macrophage, celseq2 vs fluidigm1: too few cells.

Skipping technical metric for Uncorrected_PCA, mast, celseq2 vs fluidigm1: too few cells.

Skipping technical metric for Uncorrected_PCA, quiescent_stellate, celseq2 vs fluidigm1: too few cells.

Skipping technical metric for Uncorrected_PCA, schwann, celseq2 vs fluidigm1: too few cells.

Skipping technical metric for Uncorrected_PCA, epsilon, celseq2 vs indrop: too few cells.

Skipping technical metric for Uncorrected_PCA, mast, celseq2 vs indrop: too few cells.

Skipping technical metric for Uncorrected_PCA, schwann, celseq2 vs indrop: too few cells.

Skipping technical metric for Uncorrected_PCA, epsilon, celseq2 vs smartseq2: too few cells.

Skipping technical metric for Uncorrected_PCA, macrophage, celseq2 vs smartseq2: too few cells.

Skipping technical metric for Uncorrected_PCA, mast, celseq2 vs smartseq2: too few cells.

Skipping technical metric for Uncorrected_PCA, quiescent_stellate, celseq2 vs smartseq2: too few cells.

Skipping technical metric for Uncorrected_PCA, schwann, celseq2 vs smartseq2: too few cells.

Skipping technical metric for Uncorrected_PCA, epsilon, fluidigm1 vs indrop: too few cells.

Skipping technical metric for Uncorrected_PCA, macrophage, fluidigm1 vs indrop: too few cells.

Skipping technical metric for Uncorrected_PCA, mast, fluidigm1 vs indrop: too few cells.

Skipping technical metric for Uncorrected_PCA, quiescent_stellate, fluidigm1 vs indrop: too few cells.

Skipping technical metric for Uncorrected_PCA, schwann, fluidigm1 vs indrop: too few cells.

```

## Skipping technical metric for Uncorrected_PCA, epsilon, fluidigmc1 vs smartseq2: too few cells.
## Skipping technical metric for Uncorrected_PCA, macrophage, fluidigmc1 vs smartseq2: too few cells.
## Skipping technical metric for Uncorrected_PCA, mast, fluidigmc1 vs smartseq2: too few cells.
## Skipping technical metric for Uncorrected_PCA, quiescent_stellate, fluidigmc1 vs smartseq2: too few cells.
## Skipping technical metric for Uncorrected_PCA, schwann, fluidigmc1 vs smartseq2: too few cells.
## Skipping technical metric for Uncorrected_PCA, epsilon, indrop vs smartseq2: too few cells.
## Skipping technical metric for Uncorrected_PCA, macrophage, indrop vs smartseq2: too few cells.
## Skipping technical metric for Uncorrected_PCA, mast, indrop vs smartseq2: too few cells.
## Skipping technical metric for Uncorrected_PCA, quiescent_stellate, indrop vs smartseq2: too few cells.
## Skipping technical metric for Uncorrected_PCA, schwann, indrop vs smartseq2: too few cells.
## Skipping technical metric for FastMNN, endothelial, celseq vs celseq2: too few cells.
## Skipping technical metric for FastMNN, epsilon, celseq vs celseq2: too few cells.
## Skipping technical metric for FastMNN, macrophage, celseq vs celseq2: too few cells.
## Skipping technical metric for FastMNN, mast, celseq vs celseq2: too few cells.
## Skipping technical metric for FastMNN, quiescent_stellate, celseq vs celseq2: too few cells.
## Skipping technical metric for FastMNN, schwann, celseq vs celseq2: too few cells.
## Skipping technical metric for FastMNN, endothelial, celseq vs fluidigmc1: too few cells.
## Skipping technical metric for FastMNN, epsilon, celseq vs fluidigmc1: too few cells.
## Skipping technical metric for FastMNN, macrophage, celseq vs fluidigmc1: too few cells.
## Skipping technical metric for FastMNN, mast, celseq vs fluidigmc1: too few cells.
## Skipping technical metric for FastMNN, quiescent_stellate, celseq vs fluidigmc1: too few cells.
## Skipping technical metric for FastMNN, schwann, celseq vs fluidigmc1: too few cells.
## Skipping technical metric for FastMNN, endothelial, celseq vs indrop: too few cells.
## Skipping technical metric for FastMNN, epsilon, celseq vs indrop: too few cells.

```

```

## Skipping technical metric for FastMNN, macrophage, celseq vs indrop: too few cells.

## Skipping technical metric for FastMNN, mast, celseq vs indrop: too few cells.

## Skipping technical metric for FastMNN, quiescent_stellate, celseq vs indrop: too few cells.

## Skipping technical metric for FastMNN, schwann, celseq vs indrop: too few cells.

## Skipping technical metric for FastMNN, endothelial, celseq vs smartseq2: too few cells.

## Skipping technical metric for FastMNN, epsilon, celseq vs smartseq2: too few cells.

## Skipping technical metric for FastMNN, macrophage, celseq vs smartseq2: too few cells.

## Skipping technical metric for FastMNN, mast, celseq vs smartseq2: too few cells.

## Skipping technical metric for FastMNN, quiescent_stellate, celseq vs smartseq2: too few cells.

## Skipping technical metric for FastMNN, schwann, celseq vs smartseq2: too few cells.

## Skipping technical metric for FastMNN, epsilon, celseq2 vs fluidigmc1: too few cells.

## Skipping technical metric for FastMNN, macrophage, celseq2 vs fluidigmc1: too few cells.

## Skipping technical metric for FastMNN, mast, celseq2 vs fluidigmc1: too few cells.

## Skipping technical metric for FastMNN, quiescent_stellate, celseq2 vs fluidigmc1: too few cells.

## Skipping technical metric for FastMNN, schwann, celseq2 vs fluidigmc1: too few cells.

## Skipping technical metric for FastMNN, epsilon, celseq2 vs indrop: too few cells.

## Skipping technical metric for FastMNN, mast, celseq2 vs indrop: too few cells.

## Skipping technical metric for FastMNN, schwann, celseq2 vs indrop: too few cells.

## Skipping technical metric for FastMNN, epsilon, celseq2 vs smartseq2: too few cells.

## Skipping technical metric for FastMNN, macrophage, celseq2 vs smartseq2: too few cells.

## Skipping technical metric for FastMNN, mast, celseq2 vs smartseq2: too few cells.

## Skipping technical metric for FastMNN, quiescent_stellate, celseq2 vs smartseq2: too few cells.

## Skipping technical metric for FastMNN, schwann, celseq2 vs smartseq2: too few cells.

## Skipping technical metric for FastMNN, epsilon, fluidigmc1 vs indrop: too few cells.

```

Skipping technical metric for FastMNN, macrophage, fluidigmc1 vs indrop: too few cells.

Skipping technical metric for FastMNN, mast, fluidigmc1 vs indrop: too few cells.

Skipping technical metric for FastMNN, quiescent_stellate, fluidigmc1 vs indrop: too few cells.

Skipping technical metric for FastMNN, schwann, fluidigmc1 vs indrop: too few cells.

Skipping technical metric for FastMNN, epsilon, fluidigmc1 vs smartseq2: too few cells.

Skipping technical metric for FastMNN, macrophage, fluidigmc1 vs smartseq2: too few cells.

Skipping technical metric for FastMNN, mast, fluidigmc1 vs smartseq2: too few cells.

Skipping technical metric for FastMNN, quiescent_stellate, fluidigmc1 vs smartseq2: too few cells.

Skipping technical metric for FastMNN, schwann, fluidigmc1 vs smartseq2: too few cells.

Skipping technical metric for FastMNN, epsilon, indrop vs smartseq2: too few cells.

Skipping technical metric for FastMNN, macrophage, indrop vs smartseq2: too few cells.

Skipping technical metric for FastMNN, mast, indrop vs smartseq2: too few cells.

Skipping technical metric for FastMNN, quiescent_stellate, indrop vs smartseq2: too few cells.

Skipping technical metric for FastMNN, schwann, indrop vs smartseq2: too few cells.

Skipping technical metric for MNN, endothelial, celseq vs celseq2: too few cells.

Skipping technical metric for MNN, epsilon, celseq vs celseq2: too few cells.

Skipping technical metric for MNN, macrophage, celseq vs celseq2: too few cells.

Skipping technical metric for MNN, mast, celseq vs celseq2: too few cells.

Skipping technical metric for MNN, quiescent_stellate, celseq vs celseq2: too few cells.

Skipping technical metric for MNN, schwann, celseq vs celseq2: too few cells.

Skipping technical metric for MNN, endothelial, celseq vs fluidigmc1: too few cells.

Skipping technical metric for MNN, epsilon, celseq vs fluidigmc1: too few cells.

Skipping technical metric for MNN, macrophage, celseq vs fluidigmc1: too few cells.

Skipping technical metric for MNN, mast, celseq vs fluidigmc1: too few cells.

Skipping technical metric for MNN, quiescent_stellate, celseq vs fluidigmc1: too few cells.

Skipping technical metric for MNN, schwann, celseq vs fluidigmc1: too few cells.

Skipping technical metric for MNN, endothelial, celseq vs indrop: too few cells.

Skipping technical metric for MNN, epsilon, celseq vs indrop: too few cells.

Skipping technical metric for MNN, macrophage, celseq vs indrop: too few cells.

Skipping technical metric for MNN, mast, celseq vs indrop: too few cells.

Skipping technical metric for MNN, quiescent_stellate, celseq vs indrop: too few cells.

Skipping technical metric for MNN, schwann, celseq vs indrop: too few cells.

Skipping technical metric for MNN, endothelial, celseq vs smartseq2: too few cells.

Skipping technical metric for MNN, epsilon, celseq vs smartseq2: too few cells.

Skipping technical metric for MNN, macrophage, celseq vs smartseq2: too few cells.

Skipping technical metric for MNN, mast, celseq vs smartseq2: too few cells.

Skipping technical metric for MNN, quiescent_stellate, celseq vs smartseq2: too few cells.

Skipping technical metric for MNN, schwann, celseq vs smartseq2: too few cells.

Skipping technical metric for MNN, epsilon, celseq2 vs fluidigmc1: too few cells.

Skipping technical metric for MNN, macrophage, celseq2 vs fluidigmc1: too few cells.

Skipping technical metric for MNN, mast, celseq2 vs fluidigmc1: too few cells.

Skipping technical metric for MNN, quiescent_stellate, celseq2 vs fluidigmc1: too few cells.

Skipping technical metric for MNN, schwann, celseq2 vs fluidigmc1: too few cells.

Skipping technical metric for MNN, epsilon, celseq2 vs indrop: too few cells.

Skipping technical metric for MNN, mast, celseq2 vs indrop: too few cells.

Skipping technical metric for MNN, schwann, celseq2 vs indrop: too few cells.

Skipping technical metric for MNN, epsilon, celseq2 vs smartseq2: too few cells.

Skipping technical metric for MNN, macrophage, celseq2 vs smartseq2: too few cells.

Skipping technical metric for MNN, mast, celseq2 vs smartseq2: too few cells.

Skipping technical metric for MNN, quiescent_stellate, celseq2 vs smartseq2: too few cells.

Skipping technical metric for MNN, schwann, celseq2 vs smartseq2: too few cells.

Skipping technical metric for MNN, epsilon, fluidigm1 vs indrop: too few cells.

Skipping technical metric for MNN, macrophage, fluidigm1 vs indrop: too few cells.

Skipping technical metric for MNN, mast, fluidigm1 vs indrop: too few cells.

Skipping technical metric for MNN, quiescent_stellate, fluidigm1 vs indrop: too few cells.

Skipping technical metric for MNN, schwann, fluidigm1 vs indrop: too few cells.

Skipping technical metric for MNN, epsilon, fluidigm1 vs smartseq2: too few cells.

Skipping technical metric for MNN, macrophage, fluidigm1 vs smartseq2: too few cells.

Skipping technical metric for MNN, mast, fluidigm1 vs smartseq2: too few cells.

Skipping technical metric for MNN, quiescent_stellate, fluidigm1 vs smartseq2: too few cells.

Skipping technical metric for MNN, schwann, fluidigm1 vs smartseq2: too few cells.

Skipping technical metric for MNN, epsilon, indrop vs smartseq2: too few cells.

Skipping technical metric for MNN, macrophage, indrop vs smartseq2: too few cells.

Skipping technical metric for MNN, mast, indrop vs smartseq2: too few cells.

Skipping technical metric for MNN, quiescent_stellate, indrop vs smartseq2: too few cells.

Skipping technical metric for MNN, schwann, indrop vs smartseq2: too few cells.

Skipping technical metric for Seurat_CCA, endothelial, celseq vs celseq2: too few cells.

Skipping technical metric for Seurat_CCA, epsilon, celseq vs celseq2: too few cells.

Skipping technical metric for Seurat_CCA, macrophage, celseq vs celseq2: too few cells.

Skipping technical metric for Seurat_CCA, mast, celseq vs celseq2: too few cells.

Skipping technical metric for Seurat_CCA, quiescent_stellate, celseq vs celseq2: too few cells.

Skipping technical metric for Seurat_CCA, schwann, celseq vs celseq2: too few cells.

```

## Skipping technical metric for Seurat_CCA, endothelial, celseq vs fluidigm1: too few cells.
## Skipping technical metric for Seurat_CCA, epsilon, celseq vs fluidigm1: too few cells.
## Skipping technical metric for Seurat_CCA, macrophage, celseq vs fluidigm1: too few cells.
## Skipping technical metric for Seurat_CCA, mast, celseq vs fluidigm1: too few cells.
## Skipping technical metric for Seurat_CCA, quiescent_stellate, celseq vs fluidigm1: too few cells.
## Skipping technical metric for Seurat_CCA, schwann, celseq vs fluidigm1: too few cells.
## Skipping technical metric for Seurat_CCA, endothelial, celseq vs indrop: too few cells.
## Skipping technical metric for Seurat_CCA, epsilon, celseq vs indrop: too few cells.
## Skipping technical metric for Seurat_CCA, macrophage, celseq vs indrop: too few cells.
## Skipping technical metric for Seurat_CCA, mast, celseq vs indrop: too few cells.
## Skipping technical metric for Seurat_CCA, quiescent_stellate, celseq vs indrop: too few cells.
## Skipping technical metric for Seurat_CCA, schwann, celseq vs indrop: too few cells.
## Skipping technical metric for Seurat_CCA, endothelial, celseq vs smartseq2: too few cells.
## Skipping technical metric for Seurat_CCA, epsilon, celseq vs smartseq2: too few cells.
## Skipping technical metric for Seurat_CCA, macrophage, celseq vs smartseq2: too few cells.
## Skipping technical metric for Seurat_CCA, mast, celseq vs smartseq2: too few cells.
## Skipping technical metric for Seurat_CCA, quiescent_stellate, celseq vs smartseq2: too few cells.
## Skipping technical metric for Seurat_CCA, schwann, celseq vs smartseq2: too few cells.
## Skipping technical metric for Seurat_CCA, epsilon, celseq2 vs fluidigm1: too few cells.
## Skipping technical metric for Seurat_CCA, macrophage, celseq2 vs fluidigm1: too few cells.
## Skipping technical metric for Seurat_CCA, mast, celseq2 vs fluidigm1: too few cells.
## Skipping technical metric for Seurat_CCA, quiescent_stellate, celseq2 vs fluidigm1: too few cells.
## Skipping technical metric for Seurat_CCA, schwann, celseq2 vs fluidigm1: too few cells.
## Skipping technical metric for Seurat_CCA, epsilon, celseq2 vs indrop: too few cells.

```

Skipping technical metric for Seurat_CCA, mast, celseq2 vs indrop: too few cells.

Skipping technical metric for Seurat_CCA, schwann, celseq2 vs indrop: too few cells.

Skipping technical metric for Seurat_CCA, epsilon, celseq2 vs smartseq2: too few cells.

Skipping technical metric for Seurat_CCA, macrophage, celseq2 vs smartseq2: too few cells.

Skipping technical metric for Seurat_CCA, mast, celseq2 vs smartseq2: too few cells.

Skipping technical metric for Seurat_CCA, quiescent_stellate, celseq2 vs smartseq2: too few cells.

Skipping technical metric for Seurat_CCA, schwann, celseq2 vs smartseq2: too few cells.

Skipping technical metric for Seurat_CCA, epsilon, fluidigm1 vs indrop: too few cells.

Skipping technical metric for Seurat_CCA, macrophage, fluidigm1 vs indrop: too few cells.

Skipping technical metric for Seurat_CCA, mast, fluidigm1 vs indrop: too few cells.

Skipping technical metric for Seurat_CCA, quiescent_stellate, fluidigm1 vs indrop: too few cells.

Skipping technical metric for Seurat_CCA, schwann, fluidigm1 vs indrop: too few cells.

Skipping technical metric for Seurat_CCA, epsilon, fluidigm1 vs smartseq2: too few cells.

Skipping technical metric for Seurat_CCA, macrophage, fluidigm1 vs smartseq2: too few cells.

Skipping technical metric for Seurat_CCA, mast, fluidigm1 vs smartseq2: too few cells.

Skipping technical metric for Seurat_CCA, quiescent_stellate, fluidigm1 vs smartseq2: too few cells.

Skipping technical metric for Seurat_CCA, schwann, fluidigm1 vs smartseq2: too few cells.

Skipping technical metric for Seurat_CCA, epsilon, indrop vs smartseq2: too few cells.

Skipping technical metric for Seurat_CCA, macrophage, indrop vs smartseq2: too few cells.

Skipping technical metric for Seurat_CCA, mast, indrop vs smartseq2: too few cells.

Skipping technical metric for Seurat_CCA, quiescent_stellate, indrop vs smartseq2: too few cells.

Skipping technical metric for Seurat_CCA, schwann, indrop vs smartseq2: too few cells.

```

technical_summary <- bind_rows(map(technical_results, "summary"))
technical_distance_samples <- bind_rows(map(technical_results, "distances"))

readr::write_csv(
  technical_summary,
  file.path(tables_dir, "technical_summary.csv")
)
readr::write_csv(
  technical_distance_samples,
  file.path(tables_dir, "technical_distance_samples.csv")
)

technical_summary

```

```

## # A tibble: 312 x 14
##   method batch_1 batch_2 batch_pair cell_type n_cells_batch_1 n_cells_batch_2
##   <chr>   <chr>   <chr>   <chr>   <chr>           <int>           <int>
## 1 Uncorre~ celseq  celseq2 celseq vs~ acinar             229             274
## 2 Uncorre~ celseq  celseq2 celseq vs~ activate~           19              90
## 3 Uncorre~ celseq  celseq2 celseq vs~ alpha             213            500
## 4 Uncorre~ celseq  celseq2 celseq vs~ beta             161            445
## 5 Uncorre~ celseq  celseq2 celseq vs~ delta              50            203
## 6 Uncorre~ celseq  celseq2 celseq vs~ ductal           304            257
## 7 Uncorre~ celseq  celseq2 celseq vs~ gamma             18            110
## 8 Uncorre~ celseq  fluidi~ celseq vs~ acinar             229              21
## 9 Uncorre~ celseq  fluidi~ celseq vs~ activate~           19              16
## 10 Uncorre~ celseq  fluidi~ celseq vs~ alpha             213            241
## # i 302 more rows
## # i 7 more variables: n_cross_distances <int>,
## #   n_within_distances_batch_1 <int>, n_within_distances_batch_2 <int>,
## #   median_cross_batch_distance <dbl>, within_baseline <dbl>, R_tech <dbl>,
## #   tech_deviation <dbl>

```

```

p_technical_ratio <- plot_technical_ratio_heatmap(technical_summary)
p_technical_ratio

```

Technical ratio by cell type and batch pair

R_tech = median cross-batch same-cell-type distance / within-batch baseline

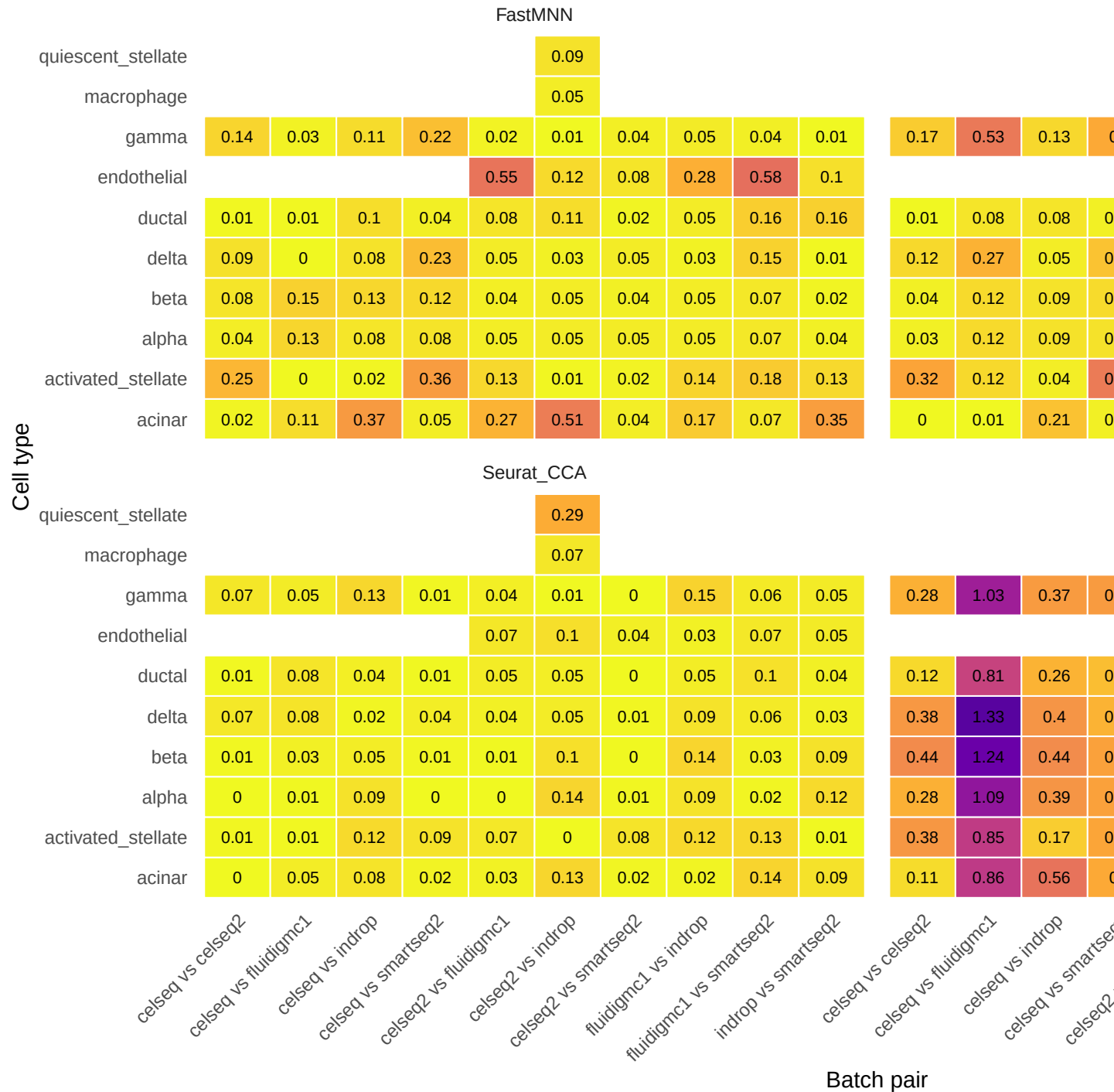
Cell type		FastMNN												
		celseq vs celseq2	celseq vs fluidigm1	celseq vs indrop	celseq vs smartseq2	celseq2 vs fluidigm1	celseq2 vs indrop	celseq2 vs smartseq2	fluidigm1 vs indrop	fluidigm1 vs smartseq2	indrop vs smartseq2	celseq vs celseq2	celseq vs fluidigm1	celseq vs indrop
quiescent_stellate	quiescent_stellate													
	macrophage													
	gamma	1.15	1.03	1.12	1.25	0.98	1.01	1.04	0.95	1.04	1.01	1.19	1.7	1.0
	endothelial					1.74	0.89	1.08	1.32	1.78	0.91			
	ductal	1.01	1.01	1.11	1.04	1.08	1.11	1.02	1.05	1.18	1.17	1.01	0.92	1.0
	delta	1.1	1	1.09	1.26	1.05	1.03	1.05	1.03	1.16	1.01	1.13	0.76	1.0
	beta	1.08	1.17	1.14	1.13	1.04	1.05	1.04	1.05	1.07	1.02	1.04	1.13	1.0
	alpha	1.04	1.14	1.08	1.08	1.05	1.05	1.05	1.05	1.07	1.04	1.03	1.13	1.0
	activated_stellate	1.29	1	1.02	1.44	1.14	1.01	1.02	0.87	1.19	0.88	1.38	0.89	0.0
	acinar	1.02	1.12	1.45	1.05	1.3	1.67	1.04	1.19	1.07	1.42	1	0.99	1.0
activated_stellate	quiescent_stellate													
	macrophage													
	gamma	1.07	0.95	0.88	1.01	0.96	1.01	1	0.86	0.94	1.06	1.32	2.8	1.0
	endothelial					1.07	0.9	1.04	0.97	0.93	1.06			
	ductal	0.99	0.92	1.04	1.01	0.95	1.05	1	0.95	0.91	1.04	1.13	2.25	1.0
	delta	1.07	0.92	0.98	1.04	0.97	1.05	1.01	0.92	0.94	1.03	1.46	3.76	1.0
	beta	1.01	1.04	1.05	1.01	1.01	1.1	1	1.15	1.03	1.09	1.55	3.47	1.0
	alpha	1	0.99	1.09	1	1	1.15	1.01	1.09	0.98	1.13	1.32	2.98	1.0
	activated_stellate	1.01	0.99	0.89	0.92	0.94	1	1.09	0.88	0.88	0.99	1.47	2.34	1.0
	acinar	1	0.95	1.09	1.02	0.97	1.14	1.02	1.02	0.87	1.09	1.12	2.36	1.0

```
save_plot(p_technical_ratio, "technical_ratio_heatmap.png", width = 12, height = 8)
```

```
p_technical_deviation <- plot_technical_deviation_heatmap(technical_summary)
p_technical_deviation
```

Technical deviation from ideal mixing

Lower abs(log(R_tech)) is better



```
save_plot(
  p_technical_deviation,
  "technical_deviation_heatmap.png",
  width = 12,
```

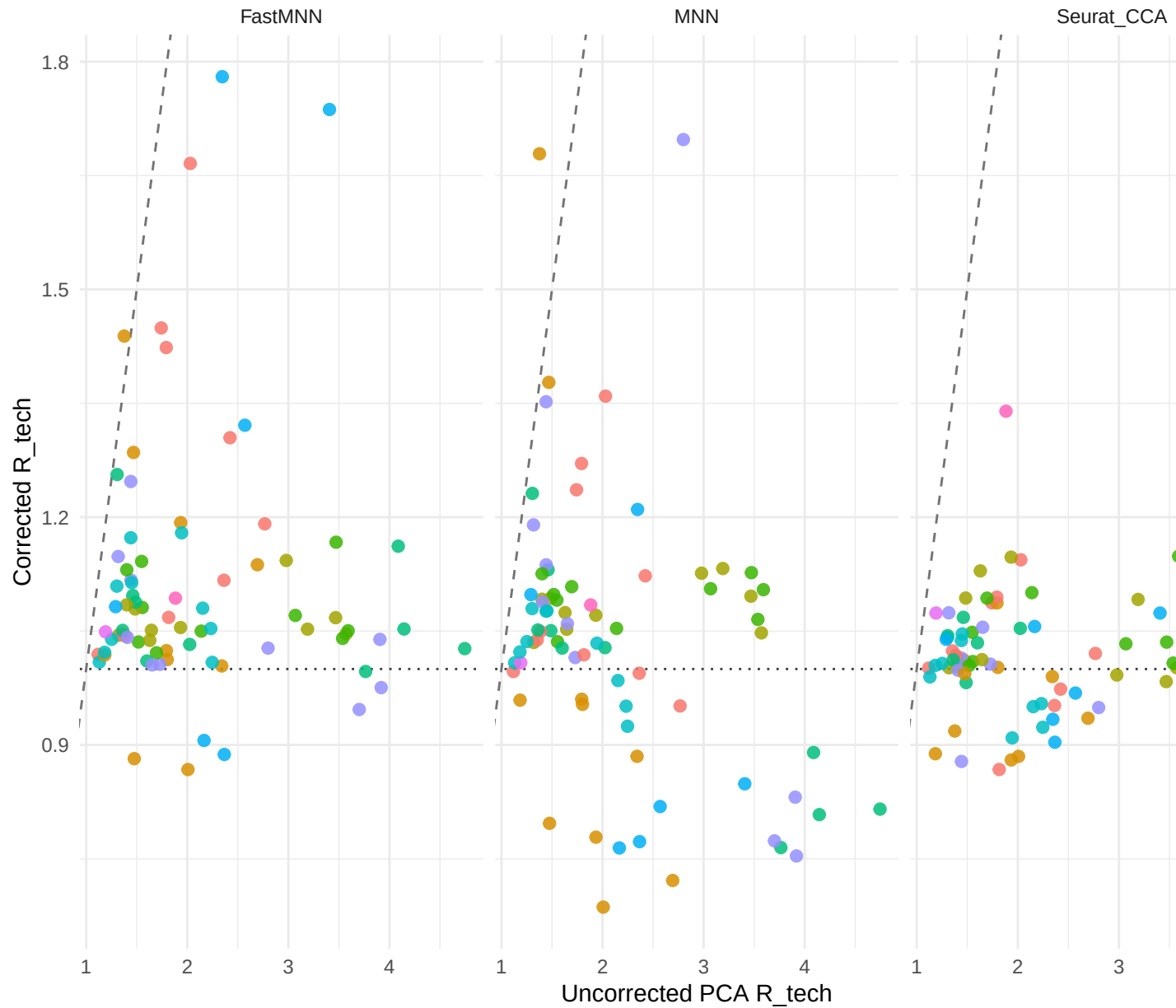
```
    height = 8
  )

technical_scatter <- make_technical_scatter_data(technical_summary)
readr::write_csv(
  technical_scatter,
  file.path(tables_dir, "technical_before_after_scatter_data.csv")
)

p_technical_scatter <- plot_technical_scatter(technical_scatter)
p_technical_scatter
```

Before-vs-after technical ratio

Each point is one cell type and batch pair



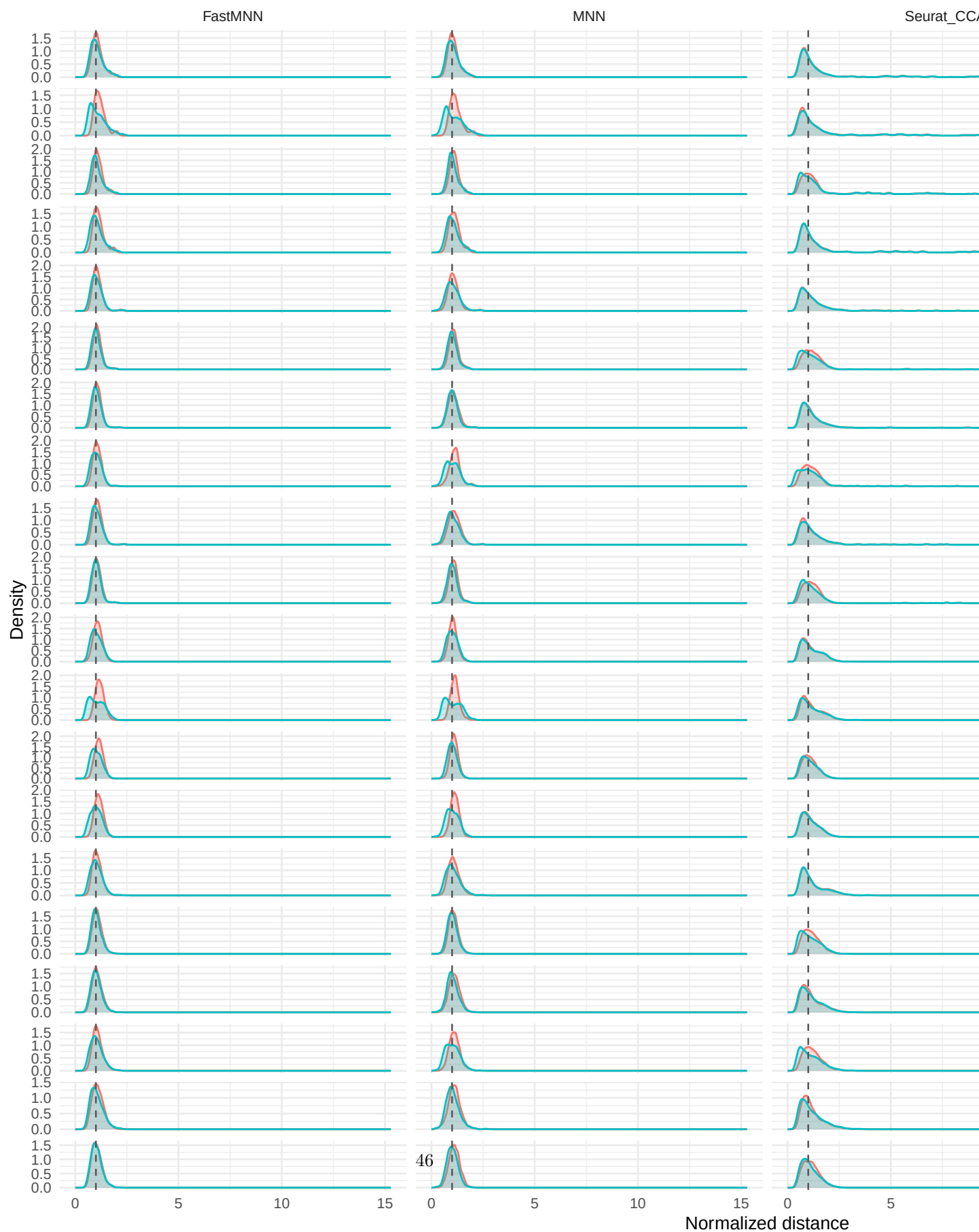
```
save_plot(p_technical_scatter, "technical_before_after_scatter.png", width = 10, height = 7)
```

```
technical_overlay <- filter_overlay_distances(
  technical_distance_samples,
  selected_batch_pairs = params$selected_overlay_batch_pairs,
  selected_cell_types = params$selected_overlay_cell_types
)

p_cross_within <- plot_cross_vs_within_overlay(technical_overlay)
p_cross_within
```


Cross-batch and within-batch normalized distance distributions

Distances are divided by the within-batch same-cell-type baseline inside each representation



```
save_plot(p_cross_within, "cross_vs_within_normalized_overlay.png", width = 13, height = 10)
```

Biological Structure Preservation Metrics

For each method, batch, and target cell type, distances are centered on the reference cell type. With `reference_cell_type = "alpha"`:

```
relative_distance = pairwise alpha-to-target distance /  
                    median alpha-to-alpha within-batch distance
```

The summary is the mean of pairwise relative distances, not a ratio of medians.

```
biological_results <- imap(embeddings, function(embedding, method_name) {  
  compute_biological_metrics_for_embedding(  
    embedding = embedding,  
    metadata = metadata,  
    batch_col = batch_col,  
    cell_type_col = cell_type_col,  
    method = method_name,  
    reference_cell_type = reference_cell_type,  
    min_cells_per_group = min_cells_per_group,  
    max_cells_per_group = max_cells_per_group,  
    max_pairs_per_comparison = max_pairs_per_comparison,  
    seed = seed  
  )  
})
```

```
## Skipping biological metric for Uncorrected_PCA, celseq, alpha vs endothelial: too few target cells.  
  
## Skipping biological metric for Uncorrected_PCA, celseq, alpha vs epsilon: too few target cells.  
  
## Skipping biological metric for Uncorrected_PCA, celseq, alpha vs macrophage: too few target cells.  
  
## Skipping biological metric for Uncorrected_PCA, celseq, alpha vs mast: too few target cells.  
  
## Skipping biological metric for Uncorrected_PCA, celseq, alpha vs quiescent_stellate: too few target  
  
## Skipping biological metric for Uncorrected_PCA, celseq, alpha vs schwann: too few target cells.  
  
## Skipping biological metric for Uncorrected_PCA, celseq2, alpha vs epsilon: too few target cells.  
  
## Skipping biological metric for Uncorrected_PCA, celseq2, alpha vs mast: too few target cells.  
  
## Skipping biological metric for Uncorrected_PCA, celseq2, alpha vs schwann: too few target cells.  
  
## Skipping biological metric for Uncorrected_PCA, fluidigm1, alpha vs epsilon: too few target cells.  
  
## Skipping biological metric for Uncorrected_PCA, fluidigm1, alpha vs macrophage: too few target cells.
```

```

## Skipping biological metric for Uncorrected_PCA, fluidigmc1, alpha vs mast: too few target cells.
## Skipping biological metric for Uncorrected_PCA, fluidigmc1, alpha vs quiescent_stellate: too few target cells.
## Skipping biological metric for Uncorrected_PCA, fluidigmc1, alpha vs schwann: too few target cells.
## Skipping biological metric for Uncorrected_PCA, smartseq2, alpha vs epsilon: too few target cells.
## Skipping biological metric for Uncorrected_PCA, smartseq2, alpha vs macrophage: too few target cells.
## Skipping biological metric for Uncorrected_PCA, smartseq2, alpha vs mast: too few target cells.
## Skipping biological metric for Uncorrected_PCA, smartseq2, alpha vs quiescent_stellate: too few target cells.
## Skipping biological metric for Uncorrected_PCA, smartseq2, alpha vs schwann: too few target cells.
## Skipping biological metric for FastMNN, celseq, alpha vs endothelial: too few target cells.
## Skipping biological metric for FastMNN, celseq, alpha vs epsilon: too few target cells.
## Skipping biological metric for FastMNN, celseq, alpha vs macrophage: too few target cells.
## Skipping biological metric for FastMNN, celseq, alpha vs mast: too few target cells.
## Skipping biological metric for FastMNN, celseq, alpha vs quiescent_stellate: too few target cells.
## Skipping biological metric for FastMNN, celseq, alpha vs schwann: too few target cells.
## Skipping biological metric for FastMNN, celseq2, alpha vs epsilon: too few target cells.
## Skipping biological metric for FastMNN, celseq2, alpha vs mast: too few target cells.
## Skipping biological metric for FastMNN, celseq2, alpha vs schwann: too few target cells.
## Skipping biological metric for FastMNN, fluidigmc1, alpha vs epsilon: too few target cells.
## Skipping biological metric for FastMNN, fluidigmc1, alpha vs macrophage: too few target cells.
## Skipping biological metric for FastMNN, fluidigmc1, alpha vs mast: too few target cells.
## Skipping biological metric for FastMNN, fluidigmc1, alpha vs quiescent_stellate: too few target cells.
## Skipping biological metric for FastMNN, fluidigmc1, alpha vs schwann: too few target cells.
## Skipping biological metric for FastMNN, smartseq2, alpha vs epsilon: too few target cells.
## Skipping biological metric for FastMNN, smartseq2, alpha vs macrophage: too few target cells.

```

Skipping biological metric for FastMNN, smartseq2, alpha vs mast: too few target cells.

Skipping biological metric for FastMNN, smartseq2, alpha vs quiescent_stellate: too few target cells.

Skipping biological metric for FastMNN, smartseq2, alpha vs schwann: too few target cells.

Skipping biological metric for MNN, celseq, alpha vs endothelial: too few target cells.

Skipping biological metric for MNN, celseq, alpha vs epsilon: too few target cells.

Skipping biological metric for MNN, celseq, alpha vs macrophage: too few target cells.

Skipping biological metric for MNN, celseq, alpha vs mast: too few target cells.

Skipping biological metric for MNN, celseq, alpha vs quiescent_stellate: too few target cells.

Skipping biological metric for MNN, celseq, alpha vs schwann: too few target cells.

Skipping biological metric for MNN, celseq2, alpha vs epsilon: too few target cells.

Skipping biological metric for MNN, celseq2, alpha vs mast: too few target cells.

Skipping biological metric for MNN, celseq2, alpha vs schwann: too few target cells.

Skipping biological metric for MNN, fluidigm1, alpha vs epsilon: too few target cells.

Skipping biological metric for MNN, fluidigm1, alpha vs macrophage: too few target cells.

Skipping biological metric for MNN, fluidigm1, alpha vs mast: too few target cells.

Skipping biological metric for MNN, fluidigm1, alpha vs quiescent_stellate: too few target cells.

Skipping biological metric for MNN, fluidigm1, alpha vs schwann: too few target cells.

Skipping biological metric for MNN, smartseq2, alpha vs epsilon: too few target cells.

Skipping biological metric for MNN, smartseq2, alpha vs macrophage: too few target cells.

Skipping biological metric for MNN, smartseq2, alpha vs mast: too few target cells.

Skipping biological metric for MNN, smartseq2, alpha vs quiescent_stellate: too few target cells.

Skipping biological metric for MNN, smartseq2, alpha vs schwann: too few target cells.

Skipping biological metric for Seurat_CCA, celseq, alpha vs endothelial: too few target cells.

Skipping biological metric for Seurat_CCA, celseq, alpha vs epsilon: too few target cells.

```
## Skipping biological metric for Seurat_CCA, celseq, alpha vs macrophage: too few target cells.
## Skipping biological metric for Seurat_CCA, celseq, alpha vs mast: too few target cells.
## Skipping biological metric for Seurat_CCA, celseq, alpha vs quiescent_stellate: too few target cells.
## Skipping biological metric for Seurat_CCA, celseq, alpha vs schwann: too few target cells.
## Skipping biological metric for Seurat_CCA, celseq2, alpha vs epsilon: too few target cells.
## Skipping biological metric for Seurat_CCA, celseq2, alpha vs mast: too few target cells.
## Skipping biological metric for Seurat_CCA, celseq2, alpha vs schwann: too few target cells.
## Skipping biological metric for Seurat_CCA, fluidigmc1, alpha vs epsilon: too few target cells.
## Skipping biological metric for Seurat_CCA, fluidigmc1, alpha vs macrophage: too few target cells.
## Skipping biological metric for Seurat_CCA, fluidigmc1, alpha vs mast: too few target cells.
## Skipping biological metric for Seurat_CCA, fluidigmc1, alpha vs quiescent_stellate: too few target cells.
## Skipping biological metric for Seurat_CCA, fluidigmc1, alpha vs schwann: too few target cells.
## Skipping biological metric for Seurat_CCA, smartseq2, alpha vs epsilon: too few target cells.
## Skipping biological metric for Seurat_CCA, smartseq2, alpha vs macrophage: too few target cells.
## Skipping biological metric for Seurat_CCA, smartseq2, alpha vs mast: too few target cells.
## Skipping biological metric for Seurat_CCA, smartseq2, alpha vs quiescent_stellate: too few target cells.
## Skipping biological metric for Seurat_CCA, smartseq2, alpha vs schwann: too few target cells.
```

```
biological_summary <- bind_rows(map(biological_results, "summary"))
biological_distance_samples <- bind_rows(map(biological_results, "distances"))
biological_delta <- compute_biological_geometry_delta(biological_summary)
```

```
readr::write_csv(
  biological_summary,
  file.path(tables_dir, "biological_summary.csv")
)
readr::write_csv(
  biological_distance_samples,
  file.path(tables_dir, "biological_distance_samples.csv")
)
readr::write_csv(
  biological_delta,
  file.path(tables_dir, "biological_signed_geometry_delta.csv")
)
```

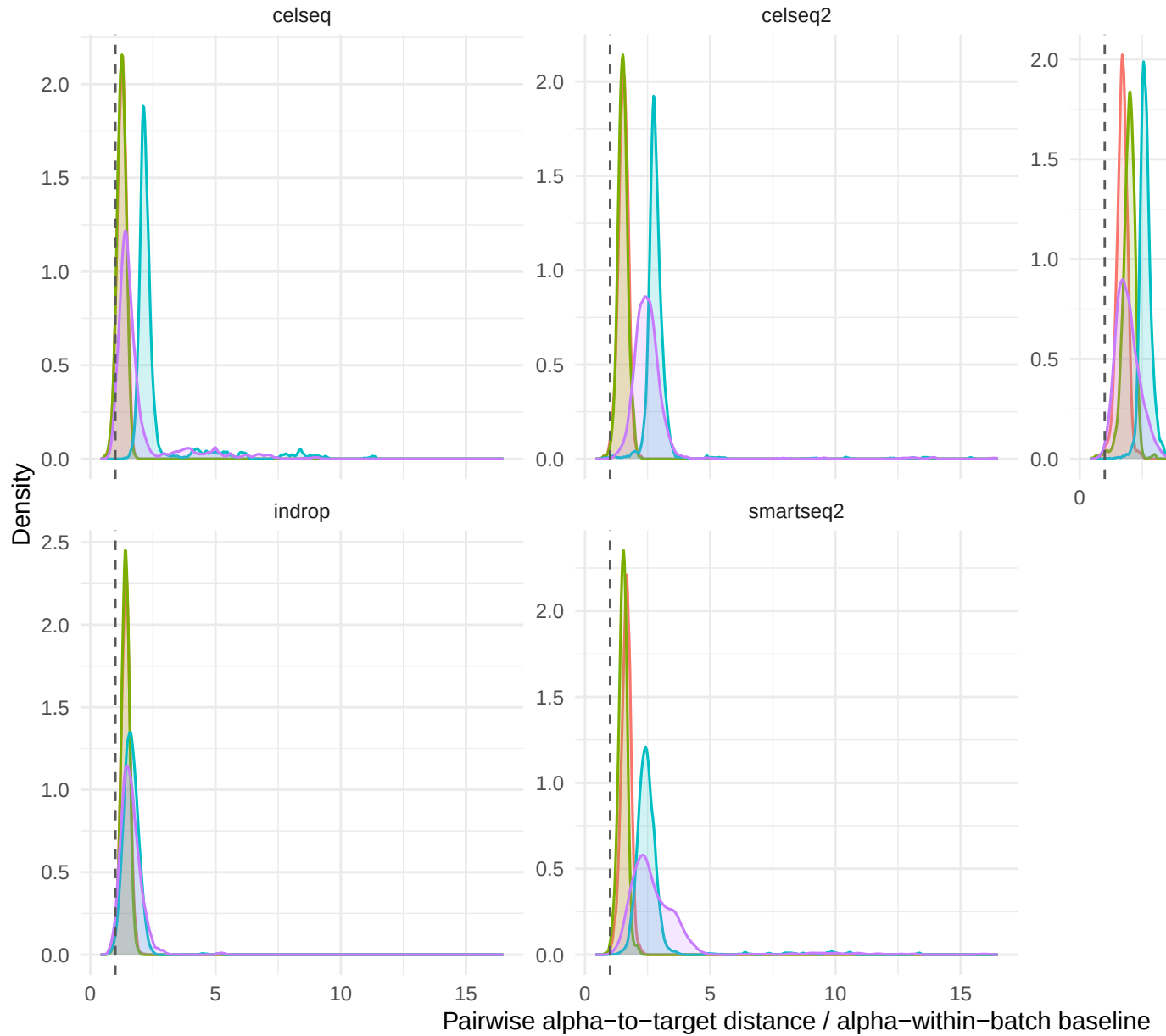
```
biological_summary
```

```
## # A tibble: 184 x 10
##   method      batch reference_cell_type target_cell_type n_reference_cells
##   <chr>      <chr>   <chr>                <chr>                <int>
## 1 Uncorrected_PCA celseq alpha          acinar                213
## 2 Uncorrected_PCA celseq alpha          activated_stell~      213
## 3 Uncorrected_PCA celseq alpha          alpha                213
## 4 Uncorrected_PCA celseq alpha          beta                 213
## 5 Uncorrected_PCA celseq alpha          delta                213
## 6 Uncorrected_PCA celseq alpha          ductal               213
## 7 Uncorrected_PCA celseq alpha          gamma                213
## 8 Uncorrected_PCA celse~ alpha          acinar                500
## 9 Uncorrected_PCA celse~ alpha          activated_stell~      500
## 10 Uncorrected_PCA celse~ alpha          alpha                500
## # i 174 more rows
## # i 5 more variables: n_target_cells <int>, n_pairwise_distances <int>,
## #   alpha_baseline <dbl>, mean_relative_separation <dbl>,
## #   median_relative_separation <dbl>
```

```
p_alpha_beta <- plot_alpha_beta_distribution(
  biological_distance_samples,
  beta_cell_type = beta_cell_type
)
p_alpha_beta
```

Relative alpha-to-beta distance distributions

Values mostly above 1 indicate separation from the alpha baseline

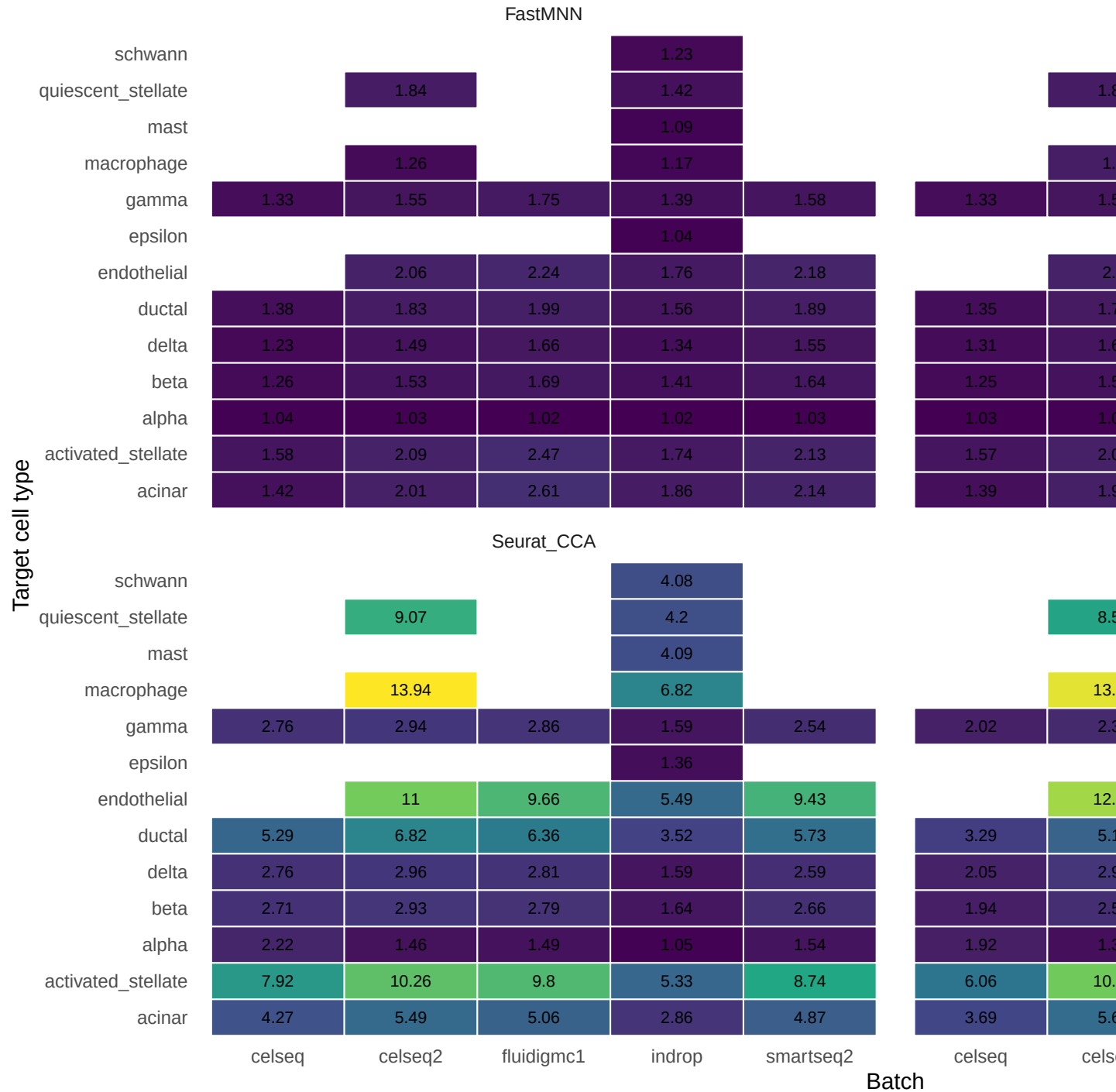


```
save_plot(p_alpha_beta, "alpha_beta_relative_distance_distribution.png", width = 11, height = 7)
```

```
p_mean_relative <- plot_mean_relative_heatmap(biological_summary)  
p_mean_relative
```

Mean alpha-centered relative separation

Mean of pairwise relative distances, not a ratio of medians



```
save_plot(p_mean_relative, "mean_relative_separation_heatmap.png", width = 12, height = 8)
```

```
biological_scatter <- make_biological_scatter_data(biological_summary)
readr::write_csv(
```

```

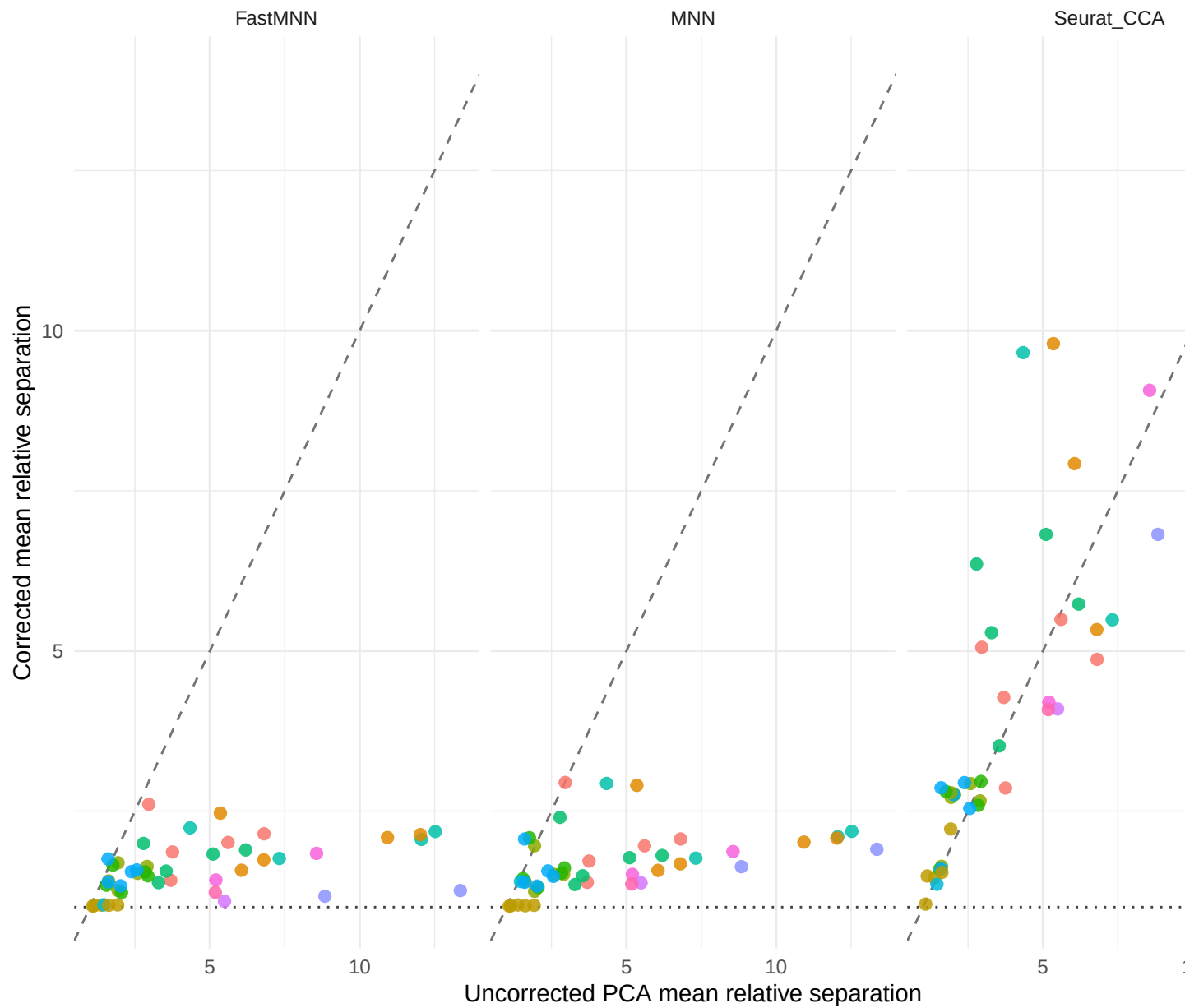
biological_scatter,
file.path(tables_dir, "biological_before_after_scatter_data.csv")
)

p_biological_scatter <- plot_biological_scatter(biological_scatter)
p_biological_scatter

```

Biological preservation relative to uncorrected PCA

Movement toward 1 can indicate biological collapse

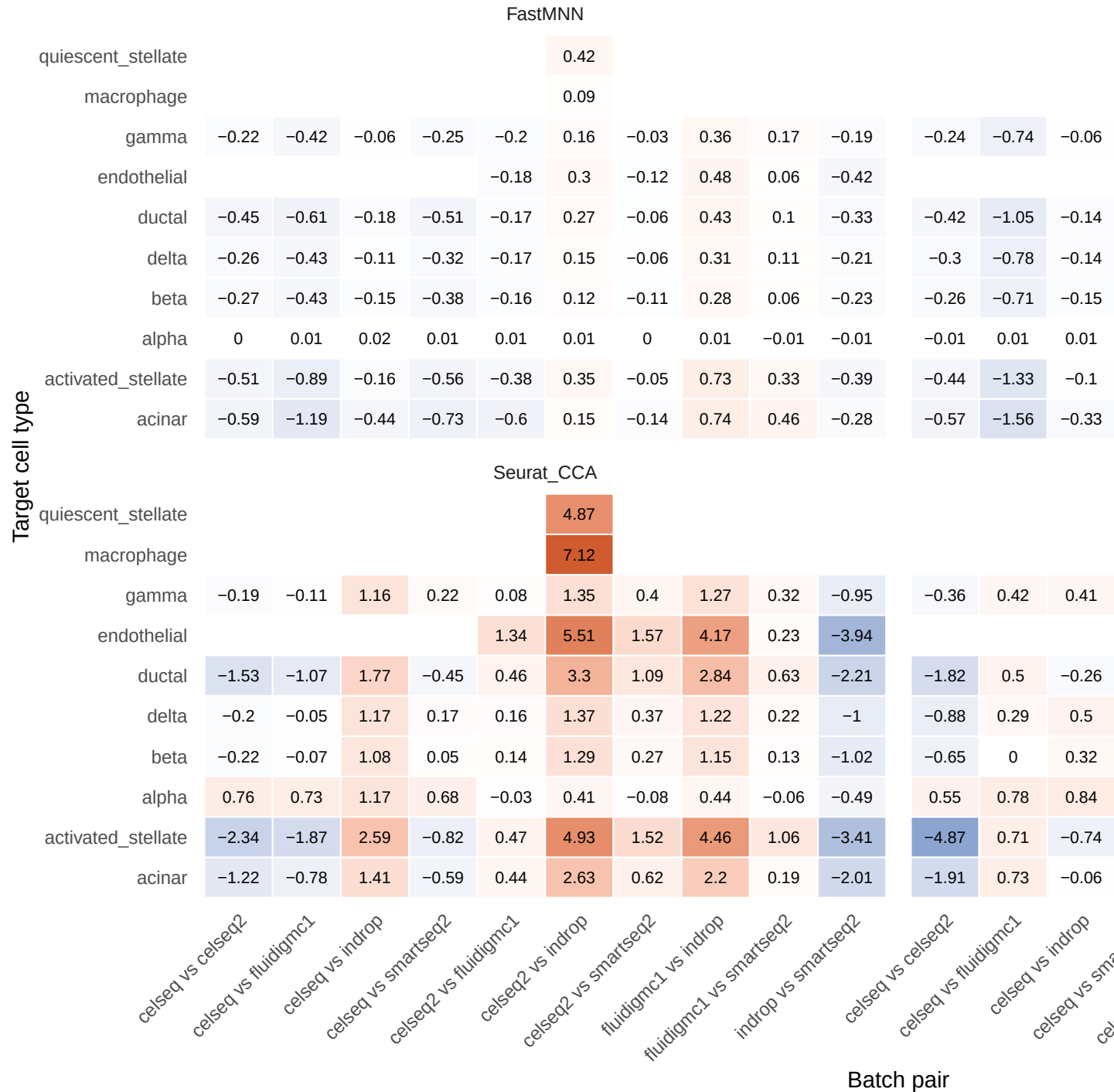


```
save_plot(p_biological_scatter, "biological_preservation_scatter.png", width = 10, height = 7)
```

```
p_signed_delta <- plot_signed_delta_heatmap(biological_delta)
p_signed_delta
```

Signed biological geometry difference across batches

Delta = mean relative separation in batch 1 minus batch 2



```
save_plot(p_signed_delta, "signed_biological_geometry_delta_heatmap.png", width = 12, height = 8)
```

Final Method Summary

```
final_method_summary <- make_final_method_summary(
  technical_summary = technical_summary,
  biological_summary = biological_summary,
  biological_delta = biological_delta,
  beta_cell_type = beta_cell_type,
  baseline_method = "Uncorrected_PCA"
)

readr::write_csv(
  final_method_summary,
  file.path(tables_dir, "final_method_summary.csv")
)

final_method_summary
```

```
## # A tibble: 4 x 10
##   method          technical_score median_R_tech n_technical_comparisons
##   <chr>                <dbl>         <dbl>                <int>
## 1 Seurat_CCA          0.0469          1.01                 78
## 2 FastMNN             0.0670          1.05                 78
## 3 MNN                 0.0904          1.05                 78
## 4 Uncorrected_PCA     0.587           1.80                 78
## # i 6 more variables: biological_preservation_score <dbl>,
## #   n_biological_comparisons <int>, biological_batch_agreement_score <dbl>,
## #   n_biological_batch_pairs <int>, alpha_beta_collapse_risk <dbl>,
## #   n_beta_batches <int>
```

Interpretation Guide

Technical Ratio Heatmap

R_{tech} compares cross-batch same-cell-type distances with same-cell-type within-batch distances in the same representation. Values near 1 indicate good mixing. Values much greater than 1 suggest undercorrection. Values much less than 1 suggest possible overcompression or overcorrection.

Technical Deviation Heatmap

This shows $\text{abs}(\log(R_{tech}))$. Lower is better because it penalizes ratios both above and below 1.

Before-vs-after Technical Scatter

Points below the $y = x$ line usually indicate reduced batch shift compared with uncorrected PCA. Points near the horizontal $y = 1$ line are closest to the target technical mixing level. Points far below 1 can be suspicious because they may indicate artificial compression.

Cross-vs-within Overlay

Cross-batch and within-batch distances are normalized by the same within-batch baseline for that method, cell type, and batch pair. Good technical correction means the cross-batch distribution overlaps the within-batch distribution.

Alpha-to-beta Relative Distribution

For the beta target, values mostly above 1 mean beta cells remain separated from the alpha baseline. If a corrected method pulls beta values toward 1, inspect it for biological collapse.

Mean Relative Separation Heatmap

Alpha should be near its own baseline. Beta and other distinct cell types should usually remain greater than 1. Missing tiles are expected when a cell type is absent or below the minimum cell-count threshold in a batch.

Biological Preservation Scatter

The x-axis is the uncorrected PCA relative separation, and the y-axis is the corrected relative separation. Movement toward 1 can indicate loss of biological geometry, especially for alpha versus beta.

Signed Biological Geometry Difference Heatmap

Values near 0 mean two batches agree about alpha-centered biological geometry. Interpret this together with the alpha-beta collapse plot: batch agreement is not useful if all methods agree because biology has been collapsed.

Final Method Summary

A useful method has a low technical score, a low biological preservation score, a low biological batch-agreement score, and an alpha-beta collapse risk safely above 1. A method with good technical score but alpha-beta collapse risk close to 1 may be overcorrecting.